Human review packet: LOS02CombinatorialAuctions

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: LOS02CombinatorialAuctions
Paper id: `LOS02CombinatorialAuctions`
Source version: not recorded
Source record: `audit/paper_statement_map.json`
Rows: 30
Generated: 2026-08-18
Source URL: official source

Review status: 0/30 source-claim screens; 0/108 library prerequisite screens. This is a review worksheet while those direct checks remain pending; it is not a final validation receipt.

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper LOS02CombinatorialAuctions --serve
```

Open `http://127.0.0.1:8765/` in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's reviewer-owned review record.

Contents

- Semantic library prerequisites (108; not paper claims)
  - Bundle Item
  - EconCSLib.Auction.Bundle
  - Allocation Agent Item
  - EconCSLib.Auction.BundleAllocation
  - EconCSLib.Auction.CombinatorialAuction
  - EconCSLib.Auction.CombinatorialReport
  - EconCSLib.Auction.CombinatorialAuction.NonnegativeValues
  - EconCSLib.Auction.CombinatorialAuction.allocation
  - EconCSLib.Auction.CombinatorialAuction.payment
  - EconCSLib.Auction.CombinatorialAuction.utility
  - EconCSLib.Auction.CombinatorialAuction.TruthfulDominantStrategy
  - EconCSLib.Auction.CombinatorialAuction.mk
  - EconCSLib.Auction.GraphCliqueDecisionInstance
  - EconCSLib.Auction.GraphCliqueDecisionInstance.graph
  - EconCSLib.Auction.GraphCliqueDecisionInstance.threshold
  - EconCSLib.Auction.GraphCliqueSelection
  - EconCSLib.Auction.GraphCliqueDecisionProblem
  - EconCSLib.Auction.GraphIndependentSetDecisionInstance
  - EconCSLib.Auction.GraphIndependentSetDecisionInstance.graph
  - EconCSLib.Auction.GraphIndependentSetDecisionInstance.mk
  - EconCSLib.Auction.GraphIndependentSetDecisionInstance.threshold
  - EconCSLib.Auction.SingleMindedBid
  - EconCSLib.Auction.SingleMindedBid.desired
  - EconCSLib.Auction.PairwiseDisjointDesired
  - EconCSLib.Auction.SetPackingFeasible
  - EconCSLib.Auction.SingleMindedAcceptedMechanism
  - EconCSLib.Auction.SingleMindedAcceptedMechanism.accepted

- EconCSLib.Auction.SingleMindedBid.mk
- EconCSLib.Auction.SingleMindedBid.value
- EconCSLib.Auction.SingleMindedAcceptedMechanism.MonotonicityOn
- EconCSLib.Auction.SingleMindedAcceptedMechanism.NonnegativeCriticalValueWithInfinityCertificate
- EconCSLib.Auction.SingleMindedAcceptedMechanism.NonnegativeCriticalValueWithInfinityCertificate.threshold
- EconCSLib.Auction.SingleMindedAcceptedMechanism.NonnegativeNonemptyProfile
- EconCSLib.Auction.SingleMindedAcceptedMechanism.payment
- EconCSLib.Auction.SingleMindedAcceptedMechanism.Participation
- EconCSLib.Auction.singleMindedAllocation
- EconCSLib.Auction.SingleMindedAcceptedMechanism.allocation
- EconCSLib.Auction.SingleMindedBid.valuation
- EconCSLib.Auction.SingleMindedAcceptedMechanism.utility
- EconCSLib.Auction.SingleMindedAcceptedMechanism.TruthfulOn
- EconCSLib.Auction.SingleMindedAcceptedMechanism.mk
- EconCSLib.Auction.singleMindedTotalValue
- EconCSLib.Auction.SingleMindedOptimalAcceptedSet
- EconCSLib.Auction.SingleMindedApproximationAtLeast
- EconCSLib.Auction.SingleMindedApproximationSolverAtLeast
- EconCSLib.Auction.SingleMindedBid.bundleSize
- EconCSLib.Auction.SingleMindedBid.averageAmountPerGood
- EconCSLib.Auction.SingleMindedAverageAmountDescending
- EconCSLib.Auction.SingleMindedBid.sqrtAmountNorm
- EconCSLib.Auction.SingleMindedOptimalSolver
- EconCSLib.Auction.SingleMindedSqrtNormDescending
- EconCSLib.Auction.SingleMindedWelfareDecisionInstance
- EconCSLib.Auction.SingleMindedWelfareDecisionInstance.bids
- EconCSLib.Auction.SingleMindedWelfareDecisionInstance.mk
- EconCSLib.Auction.SingleMindedWelfareDecisionInstance.threshold
- EconCSLib.Auction.SingleMindedWelfareDecisionProblem
- EconCSLib.Auction.weightedSetPackingValue
- EconCSLib.Auction.WeightedSetPackingOptimal
- EconCSLib.Auction.WeightedSetPackingApproximationAtLeast
- EconCSLib.Auction.WeightedSetPackingApproximationSolverAtLeast
- EconCSLib.Auction.WeightedSetPackingDecisionInstance
- EconCSLib.Auction.WeightedSetPackingDecisionInstance.mk
- EconCSLib.Auction.WeightedSetPackingDecisionInstance.sets
- EconCSLib.Auction.WeightedSetPackingDecisionInstance.threshold
- EconCSLib.Auction.WeightedSetPackingDecisionInstance.weights
- EconCSLib.Auction.WeightedSetPackingOptimalSolver
- EconCSLib.Auction.allocationValue
- EconCSLib.Auction.WelfareMaximizingAllocationRule
- EconCSLib.Auction.allocationValueExcept
- EconCSLib.Auction.reportsWithoutBidder
- EconCSLib.Auction.generalizedVickreyAuction
- EconCSLib.Auction.graphCliqueDecisionToIndependentSetComplementDecision
- EconCSLib.Auction.graphIncidentSets
- EconCSLib.Auction.graphUnitWeights
- EconCSLib.Auction.graphIndependentSetDecisionToWeightedSetPackingDecision
- EconCSLib.Auction.graphCliqueDecisionToWeightedSetPackingDecision
- EconCSLib.Auction.setPackingSingleMindedBids
- EconCSLib.Auction.setPackingDecisionToSingleMindedWelfareDecision
- EconCSLib.Auction.graphCliqueDecisionToSingleMindedWelfareDecision
- EconCSLib.Auction.setPackingSolverOfSingleMindedSolver
- EconCSLib.Auction.singleMindedAverageTieKey
- EconCSLib.Auction.singleMindedAverageTieRel
- EconCSLib.Auction.singleMindedAverageTieRelAntisymm
- EconCSLib.Auction.singleMindedAverageTieRelDecidable

  - EconCSLib.Auction.singleMindedAverageTieRelTotal
  - EconCSLib.Auction.singleMindedAverageTieRelTrans
  - EconCSLib.Auction.singleMindedAverageOrderOf
  - EconCSLib.Auction.singleMindedGreedyConflictingAccepted
  - EconCSLib.Auction.singleMindedGreedyStep
  - EconCSLib.Auction.singleMindedGreedyAcceptedFromState
  - EconCSLib.Auction.singleMindedGreedyAcceptedFromOrder
  - EconCSLib.Auction.singleMindedGreedyDeniedBecauseOfAtStateBool
  - EconCSLib.Auction.singleMindedGreedyFirstDeniedBecauseOfFromState
  - EconCSLib.Auction.singleMindedGreedyNextDeniedFromStateInOrder
  - EconCSLib.Auction.singleMindedGreedyNextDeniedFromOrder
  - EconCSLib.Auction.singleMindedGreedyPaymentFromNextDenied
  - EconCSLib.Auction.singleMindedGreedyPaymentFromOrder
  - EconCSLib.Auction.singleMindedGreedyAcceptedMechanismFromOrderOf
  - EconCSLib.Complexity.ComplexityClassModel
  - EconCSLib.Complexity.DecisionProblem
  - EconCSLib.Complexity.ComplexityClassModel.NP
  - EconCSLib.Complexity.ComplexityClassModel.ZPP
  - EconCSLib.Complexity.ComplexityClassModel.npEqZPP
  - EconCSLib.Complexity.RandomizedComplexityClassModel
  - EconCSLib.Complexity.RandomizedComplexityClassModel.RP
  - EconCSLib.Complexity.RandomizedComplexityClassModel.coNP
  - EconCSLib.Complexity.RandomizedComplexityClassModel.coRP
  - EconCSLib.Complexity.RandomizedComplexityClassModel.toComplexityClassModel
- Source claims (30)
  - utility_formulaSpec
  - truthfulOn_iffSpec
  - generalizedVickreyAuction_allocation_paymentSpec
  - singleMindedAcceptedMechanism_fieldsSpec
  - singleMindedTruthfulOn_iffSpec
  - nonnegativeNonemptySingleMindedProfile_iffSpec
  - weightedSetPackingValue_formulaSpec
  - setPackingSingleMindedBids_formulaSpec
  - averageAmountPerGood_formulaSpec
  - averageOrderOf_ruleSpec
  - greedyAcceptedFromOrder_formulaSpec
  - averageGreedyAcceptedSet_formulaSpec
  - averageGreedyPayment_formulaSpec
  - theorem4_1_generalized_vickrey_truthfulSpec
  - proposition4_2_generalized_vickrey_truthful_utility_nonnegSpec
  - theorem6_1_set_packing_feasibility_encoding_correctSpec
  - theorem6_1_set_packing_value_encoding_correctSpec
  - theorem6_1_weighted_set_packing_reductionSpec
  - theorem6_1_clique_decision_single_minded_welfare_reductionSpec
  - theorem6_1_external_optimal_solver_np_eq_zppSpec
  - theorem6_1_external_approximation_solver_np_eq_zppSpec
  - complexity_note_np_eq_zpp_implies_randomized_collapseSpec
  - theorem7_2_sqrt_norm_approx_of_sorted_orderSpec
  - lemma9_1_exists_nonnegative_critical_value_of_monotonicitySpec
  - lemma9_2_denied_bidder_utility_eq_zeroSpec
  - lemma9_3_truthful_utility_nonnegative_conditionSpec
  - lemma9_4_no_profitable_value_only_lie_of_nonnegative_infinity_axiomsSpec
  - lemma9_5_finite_threshold_mono_of_nonnegative_infinity_certificateSpec
  - theorem9_6_single_minded_truthful_of_nonnegative_infinity_axiomsSpec
  - theorem10_2_averageGreedy_truthfulSpec

Material library prerequisites

`Bundle Item`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(Item : Type u_1) → Finset Item
```

Exact Lean library declaration

```
/-- A bundle of indivisible goods. -/
abbrev Bundle (Item : Type*) := Finset Item
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.Bundle`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(Item : Type u_1) → EconCSLib.FairDivision.Bundle Item
```

Exact Lean library declaration

```
abbrev Bundle (Item : Type*) := FairDivision.Bundle Item
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`Allocation Agent Item`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(Agent : Type u_1) → (Item : Type u_2) → Agent → EconCSLib.FairDivision.Bundle Item
```

Exact Lean library declaration

```
/-- An allocation assigns each agent a bundle. -/
abbrev Allocation (Agent Item : Type*) := Agent → Bundle Item
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.BundleAllocation`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(Bidder : Type u_1) → (Item : Type u_2) → EconCSLib.FairDivision.Allocation Bidder Item
```

Exact Lean library declaration

```
abbrev BundleAllocation (Bidder Item : Type*) :=
  FairDivision.Allocation Bidder Item
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.CombinatorialAuction`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
Type u_1 → Type u_2 → Type (max u_1 u_2)
```

Exact Lean library declaration

```
structure CombinatorialAuction (Bidder Item : Type*) where
  allocation : CombinatorialReport Bidder Item → BundleAllocation Bidder Item
  payment : CombinatorialReport Bidder Item → Bidder → ℝ
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.CombinatorialReport`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(Bidder : Type u_1) → (Item : Type u_2) → Bidder → EconCSLib.Auction.Bundle Item → ℝ
```

Exact Lean library declaration

```
abbrev CombinatorialReport (Bidder Item : Type*) :=
  Bidder → Bundle Item → ℝ
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.CombinatorialAuction.NonnegativeValues`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} (values : EconCSLib.Auction.CombinatorialReport Bidder Item) (i : Bidder)
  (S : EconCSLib.Auction.Bundle Item), 0 ≤ values i S
```

Exact Lean library declaration

```
def NonnegativeValues (values : CombinatorialReport Bidder Item) : Prop :=
  ∀ i S, 0 ≤ values i S
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.CombinatorialAuction.allocation`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    (self : EconCSLib.Auction.CombinatorialAuction Bidder Item) →
      (a : EconCSLib.Auction.CombinatorialReport Bidder Item) → (a_1 : Bidder) → self.1 a a_1
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.CombinatorialAuction.payment`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    (self : EconCSLib.Auction.CombinatorialAuction Bidder Item) →
      (a : EconCSLib.Auction.CombinatorialReport Bidder Item) → (a_1 : Bidder) → self.2 a a_1
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.CombinatorialAuction.utility`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    (M : EconCSLib.Auction.CombinatorialAuction Bidder Item) →
      (values reports : EconCSLib.Auction.CombinatorialReport Bidder Item) →
        (i : Bidder) → values i (M.allocation reports i) - M.payment reports i
```

Exact Lean library declaration

```
def utility (M : CombinatorialAuction Bidder Item)
    (values reports : CombinatorialReport Bidder Item) (i : Bidder) : ℝ :=
  values i (M.allocation reports i) - M.payment reports i
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.CombinatorialAuction.TruthfulDominantStrategy`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder]
  (M : EconCSLib.Auction.CombinatorialAuction Bidder Item) (values : EconCSLib.Auction.CombinatorialReport Bidder Item)
  (i : Bidder) (report : EconCSLib.Auction.Bundle Item → ℝ),
  M.utility values (Function.update values i report) i ≤ M.utility values values i
```

Exact Lean library declaration

```
def TruthfulDominantStrategy [DecidableEq Bidder]
    (M : CombinatorialAuction Bidder Item) : Prop :=
  ∀ (values : CombinatorialReport Bidder Item)
      (i : Bidder) (report : Bundle Item → ℝ),
    M.utility values (Function.update values i report) i ≤
      M.utility values values i
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.CombinatorialAuction.mk`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    (EconCSLib.Auction.CombinatorialReport Bidder Item → EconCSLib.Auction.BundleAllocation Bidder Item) →
      (EconCSLib.Auction.CombinatorialReport Bidder Item → Bidder → ℝ) →
        EconCSLib.Auction.CombinatorialAuction Bidder Item
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.GraphCliqueDecisionInstance`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
Type u_4 → Type u_4
```

Exact Lean library declaration

```
structure GraphCliqueDecisionInstance (Vertex : Type*) where
  graph : SimpleGraph Vertex
  threshold : ℝ
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.GraphCliqueDecisionInstance.graph`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Vertex : Type u_4} → (self : EconCSLib.Auction.GraphCliqueDecisionInstance Vertex) → self.1
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.GraphCliqueDecisionInstance.threshold`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Vertex : Type u_4} → (self : EconCSLib.Auction.GraphCliqueDecisionInstance Vertex) → self.2
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.GraphCliqueSelection`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Vertex : Type u_3} (G : SimpleGraph Vertex) (selected : Finset Vertex) {{u v : Vertex}},
  u ∈ selected → v ∈ selected → u ≠ v → G.Adj u v
```

Exact Lean library declaration

```
def GraphCliqueSelection
    (G : SimpleGraph Vertex) (selected : Finset Vertex) : Prop :=
  ∀ {{u v : Vertex}},
    u ∈ selected → v ∈ selected → u ≠ v → G.Adj u v
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.GraphCliqueDecisionProblem`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Vertex : Type u_3} (a : EconCSLib.Auction.GraphCliqueDecisionInstance Vertex),
  ∃ selected, EconCSLib.Auction.GraphCliqueSelection a.graph selected ∧ a.threshold ≤ ↑selected.card
```

Exact Lean library declaration

```
def GraphCliqueDecisionProblem :
    EconCSLib.Complexity.DecisionProblem
      (GraphCliqueDecisionInstance Vertex) :=
  fun problem =>
    ∃ selected,
      GraphCliqueSelection problem.graph selected ∧
        problem.threshold ≤ (selected.card : ℝ)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.GraphIndependentSetDecisionInstance`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
Type u_4 → Type u_4
```

Exact Lean library declaration

```
structure GraphIndependentSetDecisionInstance (Vertex : Type*) where
  graph : SimpleGraph Vertex
  threshold : ℝ
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.GraphIndependentSetDecisionInstance.graph`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Vertex : Type u_4} → (self : EconCSLib.Auction.GraphIndependentSetDecisionInstance Vertex) → self.1
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.GraphIndependentSetDecisionInstance.mk`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Vertex : Type u_4} → SimpleGraph Vertex → ℝ → EconCSLib.Auction.GraphIndependentSetDecisionInstance Vertex
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

`EconCSLib.Auction.GraphIndependentSetDecisionInstance.threshold`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Vertex : Type u_4} → (self : EconCSLib.Auction.GraphIndependentSetDecisionInstance Vertex) → self.2
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedBid`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
Type u_3 → Type u_3
```

Exact Lean library declaration

```
structure SingleMindedBid (Item : Type*) where
  desired : Bundle Item
  value : ℝ
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedBid.desired`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Item : Type u_3} → (self : EconCSLib.Auction.SingleMindedBid Item) → self.1
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.PairwiseDisjointDesired`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [DecidableEq Bidder] [DecidableEq Item]
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) (accepted : Finset Bidder) {{i j : Bidder}},
  i ∈ accepted → j ∈ accepted → i ≠ j → Disjoint (bids i).desired (bids j).desired
```

Exact Lean library declaration

```
def PairwiseDisjointDesired [DecidableEq Bidder]
    [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item) (accepted : Finset Bidder) : Prop :=
  ∀ {{i j : Bidder}},
    i ∈ accepted → j ∈ accepted → i ≠ j →
      Disjoint (bids i).desired (bids j).desired
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SetPackingFeasible`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [DecidableEq Bidder] [DecidableEq Item] (sets : Bidder → Finset Item)
  (selected : Finset Bidder) {{i j : Bidder}}, i ∈ selected → j ∈ selected → i ≠ j → Disjoint (sets i) (sets j)
```

Exact Lean library declaration

```
def SetPackingFeasible [DecidableEq Bidder] [DecidableEq Item]
    (sets : Bidder → Finset Item) (selected : Finset Bidder) : Prop :=
  ∀ {{i j : Bidder}},
    i ∈ selected → j ∈ selected → i ≠ j →
      Disjoint (sets i) (sets j)
```

Recorded source-to-library screening **Verdict:** not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
Type u_3 → Type u_4 → Type (max u_3 u_4)
```

Exact Lean library declaration

```
structure SingleMindedAcceptedMechanism (Bidder Item : Type*) where
  accepted : (Bidder → SingleMindedBid Item) → Finset Bidder
  payment : (Bidder → SingleMindedBid Item) → Bidder → ℝ
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.accepted`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    (self : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item) →
      (a : Bidder → EconCSLib.Auction.SingleMindedBid Item) → self.1 a
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedBid.mk`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Item : Type u_3} → EconCSLib.Auction.Bundle Item → ℝ → EconCSLib.Auction.SingleMindedBid Item
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

`EconCSLib.Auction.SingleMindedBid.value`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Item : Type u_3} → (self : EconCSLib.Auction.SingleMindedBid Item) → self.2
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.MonotonicityOn`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_3} {Item : Type u_4} [inst : DecidableEq Bidder]
  (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item)
  (admissible : (Bidder → EconCSLib.Auction.SingleMindedBid Item) → Prop)
  (reports : Bidder → EconCSLib.Auction.SingleMindedBid Item),
  admissible reports →
    ∀ (i : Bidder) (s' : EconCSLib.Auction.Bundle Item) (v' : ℝ),
      i ∈ M.accepted reports →
        admissible (Function.update reports i { desired := s', value := v' }) →
          s' ⊆ (reports i).desired →
            (reports i).value ≤ v' → i ∈ M.accepted (Function.update reports i { desired := s', value := v' })
```

Exact Lean library declaration

```
def MonotonicityOn [DecidableEq Bidder]
    (M : SingleMindedAcceptedMechanism Bidder Item)
    (admissible : (Bidder → SingleMindedBid Item) → Prop) : Prop :=
  ∀ reports,
    admissible reports →
      ∀ i s' v',
        i ∈ M.accepted reports →
          admissible (Function.update reports i { desired := s', value := v' }) →
            s' ⊆ (reports i).desired →
              (reports i).value ≤ v' →
                i ∈ M.accepted
                  (Function.update reports i { desired := s', value := v' })
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.NonnegativeCriticalValueWithInfinityCertificate`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    [DecidableEq Bidder] →
      [DecidableEq Item] → EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item → Type (max u_3 u_4)
```

Exact Lean library declaration

```
structure NonnegativeCriticalValueWithInfinityCertificate
    [DecidableEq Bidder] [DecidableEq Item]
    (M : SingleMindedAcceptedMechanism Bidder Item) where
  threshold : (Bidder → SingleMindedBid Item) → Bidder → Bundle Item → Option ℝ
  threshold_nonneg :
    ∀ reports,
      NonnegativeNonemptyProfile reports →
        ∀ i s p, s.Nonempty → threshold reports i s = some p → 0 ≤ p
  threshold_own_value_independent :
    ∀ reports,
      NonnegativeNonemptyProfile reports →
        ∀ i s v,
          s.Nonempty →
            0 ≤ v →
              threshold
                  (Function.update reports i { desired := s, value := v }) i s =
                threshold reports i s
  below_denied :
    ∀ reports,
      NonnegativeNonemptyProfile reports →
        ∀ i s v p,
          s.Nonempty →
            0 ≤ v →
              threshold reports i s = some p →
                v < p →
                  i ∉ M.accepted
                    (Function.update reports i { desired := s, value := v })
  above_granted :
    ∀ reports,
      NonnegativeNonemptyProfile reports →
        ∀ i s v p,
          s.Nonempty →
            0 ≤ v →
              threshold reports i s = some p →
                p < v →
                  i ∈ M.accepted
                    (Function.update reports i { desired := s, value := v })
  infinite_denied :
    ∀ reports,
      NonnegativeNonemptyProfile reports →
        ∀ i s v,
          s.Nonempty →
            0 ≤ v →
              threshold reports i s = none →
                i ∉ M.accepted
                  (Function.update reports i { desired := s, value := v })
  payment_eq_of_accepted :
    ∀ reports,
      NonnegativeNonemptyProfile reports →
        ∀ i,
          i ∈ M.accepted reports →
            ∃ p,
              threshold reports i (reports i).desired = some p ∧
                M.payment reports i = p
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.NonnegativeCriticalValueWithInfinityCertificate.threshold`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        {M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item} →
          (self : M.NonnegativeCriticalValueWithInfinityCertificate) →
            (a : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
              (a_1 : Bidder) → (a_2 : EconCSLib.Auction.Bundle Item) → self.1 a a_1 a_2
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.NonnegativeNonemptyProfile`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_3} {Item : Type u_4} [DecidableEq Item] (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item)
  (i : Bidder), Finset.Nonempty (bids i).desired ∧ 0 ≤ (bids i).value
```

Exact Lean library declaration

```
def NonnegativeNonemptyProfile [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item) : Prop :=
  ∀ i, (bids i).desired.Nonempty ∧ 0 ≤ (bids i).value
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.payment`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    (self : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item) →
      (a : Bidder → EconCSLib.Auction.SingleMindedBid Item) → (a_1 : Bidder) → self.2 a a_1
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.Participation`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_3} {Item : Type u_4} (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item)
  (reports : Bidder → EconCSLib.Auction.SingleMindedBid Item), ∀ i ∉ M.accepted reports, M.payment reports i = 0
```

Exact Lean library declaration

```
def Participation (M : SingleMindedAcceptedMechanism Bidder Item) : Prop :=
  ∀ reports i, i ∉ M.accepted reports → M.payment reports i = 0
```

Recorded source-to-library screening **Verdict:** not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedAllocation`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
        (accepted : Finset Bidder) → (a : Bidder) → if a ∈ accepted then (bids a).desired else ∅
```

Exact Lean library declaration

```
def singleMindedAllocation [DecidableEq Bidder]
    (bids : Bidder → SingleMindedBid Item) (accepted : Finset Bidder) :
    BundleAllocation Bidder Item :=
  fun i => if i ∈ accepted then (bids i).desired else ∅
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.allocation`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    [inst : DecidableEq Bidder] →
      (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item) →
        (reports : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (a : Bidder) → EconCSLib.Auction.singleMindedAllocation reports (M.accepted reports) a
```

Exact Lean library declaration

```
def allocation [DecidableEq Bidder]
    (M : SingleMindedAcceptedMechanism Bidder Item)
    (reports : Bidder → SingleMindedBid Item) :
    BundleAllocation Bidder Item :=
  singleMindedAllocation reports (M.accepted reports)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedBid.valuation`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Item : Type u_2} →
  [inst : DecidableEq Item] →
    (b : EconCSLib.Auction.SingleMindedBid Item) →
      (a : EconCSLib.Auction.Bundle Item) → if b.desired ⊆ a then b.value else 0
```

Exact Lean library declaration

```
def valuation (b : SingleMindedBid Item) : Bundle Item → ℝ :=
  fun S => if b.desired ⊆ S then b.value else 0
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.utility`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item) →
          (values reports : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
            (i : Bidder) → (values i).valuation (M.allocation reports i) - M.payment reports i
```

Exact Lean library declaration

```
noncomputable def utility [DecidableEq Bidder] [DecidableEq Item]
    (M : SingleMindedAcceptedMechanism Bidder Item)
    (values reports : Bidder → SingleMindedBid Item) (i : Bidder) : ℝ :=
  (values i).valuation (M.allocation reports i) - M.payment reports i
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.TruthfulOn`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_3} {Item : Type u_4} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item)
  (admissible : (Bidder → EconCSLib.Auction.SingleMindedBid Item) → Prop)
  (values : Bidder → EconCSLib.Auction.SingleMindedBid Item),
  admissible values →
    ∀ (i : Bidder) (report : EconCSLib.Auction.SingleMindedBid Item),
      admissible (Function.update values i report) →
        M.utility values (Function.update values i report) i ≤ M.utility values values i
```

Exact Lean library declaration

```
def TruthfulOn [DecidableEq Bidder] [DecidableEq Item]
    (M : SingleMindedAcceptedMechanism Bidder Item)
    (admissible : (Bidder → SingleMindedBid Item) → Prop) : Prop :=
  ∀ values,
    admissible values →
      ∀ i report,
        admissible (Function.update values i report) →
          M.utility values (Function.update values i report) i ≤
            M.utility values values i
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAcceptedMechanism.mk`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    ((Bidder → EconCSLib.Auction.SingleMindedBid Item) → Finset Bidder) →
      ((Bidder → EconCSLib.Auction.SingleMindedBid Item) → Bidder → ℝ) →
        EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedTotalValue`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [DecidableEq Bidder] →
      (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
        (selected : Finset Bidder) → ∑ i ∈ selected, (bids i).value
```

Exact Lean library declaration

```
noncomputable def singleMindedTotalValue [DecidableEq Bidder]
    (bids : Bidder → SingleMindedBid Item) (selected : Finset Bidder) : ℝ :=
  ∑ i ∈ selected, (bids i).value
```

Recorded source-to-library screening **Verdict:** not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedOptimalAcceptedSet`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) (selected : Finset Bidder),
  EconCSLib.Auction.PairwiseDisjointDesired bids selected ∧
    ∀ (other : Finset Bidder),
      EconCSLib.Auction.PairwiseDisjointDesired bids other →
        EconCSLib.Auction.singleMindedTotalValue bids other ≤ EconCSLib.Auction.singleMindedTotalValue bids selected
```

Exact Lean library declaration

```
def SingleMindedOptimalAcceptedSet [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item) (selected : Finset Bidder) : Prop :=
  PairwiseDisjointDesired bids selected ∧
    ∀ other, PairwiseDisjointDesired bids other →
      singleMindedTotalValue bids other ≤ singleMindedTotalValue bids selected
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedApproximationAtLeast`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) (factor : ℝ) (selected : Finset Bidder),
  EconCSLib.Auction.PairwiseDisjointDesired bids selected ∧
    ∀ (optimal : Finset Bidder),
      EconCSLib.Auction.SingleMindedOptimalAcceptedSet bids optimal →
        factor * EconCSLib.Auction.singleMindedTotalValue bids optimal ≤
          EconCSLib.Auction.singleMindedTotalValue bids selected
```

Exact Lean library declaration

```
def SingleMindedApproximationAtLeast
    [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (factor : ℝ) (selected : Finset Bidder) : Prop :=
  PairwiseDisjointDesired bids selected ∧
    ∀ optimal, SingleMindedOptimalAcceptedSet bids optimal →
      factor * singleMindedTotalValue bids optimal ≤
        singleMindedTotalValue bids selected
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedApproximationSolverAtLeast`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item] (factor : ℝ)
  (solver : (Bidder → EconCSLib.Auction.SingleMindedBid Item) → Finset Bidder)
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item),
  EconCSLib.Auction.SingleMindedApproximationAtLeast bids factor (solver bids)
```

Exact Lean library declaration

```
def SingleMindedApproximationSolverAtLeast
    [DecidableEq Bidder] [DecidableEq Item]
    (factor : ℝ)
    (solver : (Bidder → SingleMindedBid Item) → Finset Bidder) : Prop :=
  ∀ bids, SingleMindedApproximationAtLeast bids factor (solver bids)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedBid.bundleSize`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Item : Type u_2} → (b : EconCSLib.Auction.SingleMindedBid Item) → ↑(Finset.card b.desired)
```

Exact Lean library declaration

```
def bundleSize (b : SingleMindedBid Item) : ℝ :=
  (b.desired.card : ℝ)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedBid.averageAmountPerGood`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Item : Type u_2} → (b : EconCSLib.Auction.SingleMindedBid Item) → b.value / b.bundleSize
```

Exact Lean library declaration

```
noncomputable def averageAmountPerGood (b : SingleMindedBid Item) : ℝ :=
  b.value / b.bundleSize
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedAverageAmountDescending`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [DecidableEq Item] (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item)
  (order : List Bidder),
  List.Pairwise (fun earlier later => (bids later).averageAmountPerGood ≤ (bids earlier).averageAmountPerGood) order
```

Exact Lean library declaration

```
def SingleMindedAverageAmountDescending [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item) (order : List Bidder) : Prop :=
  order.Pairwise fun earlier later =>
    (bids later).averageAmountPerGood ≤ (bids earlier).averageAmountPerGood
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedBid.sqrtAmountNorm`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Item : Type u_2} → (b : EconCSLib.Auction.SingleMindedBid Item) → b.value / √b.bundleSize
```

Exact Lean library declaration

```
noncomputable def sqrtAmountNorm (b : SingleMindedBid Item) : ℝ :=
  b.value / Real.sqrt b.bundleSize
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedOptimalSolver`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (solver : (Bidder → EconCSLib.Auction.SingleMindedBid Item) → Finset Bidder)
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item),
  EconCSLib.Auction.SingleMindedOptimalAcceptedSet bids (solver bids)
```

Exact Lean library declaration

```
def SingleMindedOptimalSolver
    [DecidableEq Bidder] [DecidableEq Item]
    (solver : (Bidder → SingleMindedBid Item) → Finset Bidder) : Prop :=
  ∀ bids, SingleMindedOptimalAcceptedSet bids (solver bids)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedSqrtNormDescending`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [DecidableEq Item] (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item)
  (order : List Bidder),
  List.Pairwise (fun earlier later => (bids later).sqrtAmountNorm ≤ (bids earlier).sqrtAmountNorm) order
```

Exact Lean library declaration

```
def SingleMindedSqrtNormDescending [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item) (order : List Bidder) : Prop :=
  order.Pairwise fun earlier later =>
    (bids later).sqrtAmountNorm ≤ (bids earlier).sqrtAmountNorm
```

Recorded source-to-library screening **Verdict:** not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedWelfareDecisionInstance`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
Type u_3 → Type u_4 → Type (max u_3 u_4)
```

Exact Lean library declaration

```
structure SingleMindedWelfareDecisionInstance (Bidder Item : Type*) where
  bids : Bidder → SingleMindedBid Item
  threshold : ℝ
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedWelfareDecisionInstance.bids`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    (self : EconCSLib.Auction.SingleMindedWelfareDecisionInstance Bidder Item) → (a : Bidder) → self.1 a
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedWelfareDecisionInstance.mk`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    (Bidder → EconCSLib.Auction.SingleMindedBid Item) →
      ℝ → EconCSLib.Auction.SingleMindedWelfareDecisionInstance Bidder Item
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedWelfareDecisionInstance.threshold`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} → (self : EconCSLib.Auction.SingleMindedWelfareDecisionInstance Bidder Item) → self.2
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.SingleMindedWelfareDecisionProblem`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (a : EconCSLib.Auction.SingleMindedWelfareDecisionInstance Bidder Item),
  ∃ selected,
    EconCSLib.Auction.PairwiseDisjointDesired a.bids selected ∧
      a.threshold ≤ EconCSLib.Auction.singleMindedTotalValue a.bids selected
```

Exact Lean library declaration

```
noncomputable def SingleMindedWelfareDecisionProblem
    [DecidableEq Bidder] [DecidableEq Item] :
    EconCSLib.Complexity.DecisionProblem
      (SingleMindedWelfareDecisionInstance Bidder Item) :=
  fun problem =>
    ∃ selected,
      PairwiseDisjointDesired problem.bids selected ∧
        problem.threshold ≤ singleMindedTotalValue problem.bids selected
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.weightedSetPackingValue`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  [DecidableEq Bidder] → (weights : Bidder → ℝ) → (selected : Finset Bidder) → ∑ i ∈ selected, weights i
```

Exact Lean library declaration

```
noncomputable def weightedSetPackingValue [DecidableEq Bidder]
    (weights : Bidder → ℝ) (selected : Finset Bidder) : ℝ :=
  ∑ i ∈ selected, weights i
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WeightedSetPackingOptimal`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (sets : Bidder → Finset Item) (weights : Bidder → ℝ) (selected : Finset Bidder),
  EconCSLib.Auction.SetPackingFeasible sets selected ∧
    ∀ (other : Finset Bidder),
      EconCSLib.Auction.SetPackingFeasible sets other →
        EconCSLib.Auction.weightedSetPackingValue weights other ≤
          EconCSLib.Auction.weightedSetPackingValue weights selected
```

Exact Lean library declaration

```
def WeightedSetPackingOptimal [DecidableEq Bidder] [DecidableEq Item]
    (sets : Bidder → Finset Item) (weights : Bidder → ℝ)
    (selected : Finset Bidder) : Prop :=
  SetPackingFeasible sets selected ∧
    ∀ other, SetPackingFeasible sets other →
      weightedSetPackingValue weights other ≤ weightedSetPackingValue weights selected
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WeightedSetPackingApproximationAtLeast`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (sets : Bidder → Finset Item) (weights : Bidder → ℝ) (factor : ℝ) (selected : Finset Bidder),
  EconCSLib.Auction.SetPackingFeasible sets selected ∧
    ∀ (optimal : Finset Bidder),
      EconCSLib.Auction.WeightedSetPackingOptimal sets weights optimal →
        factor * EconCSLib.Auction.weightedSetPackingValue weights optimal ≤
          EconCSLib.Auction.weightedSetPackingValue weights selected
```

Exact Lean library declaration

```
def WeightedSetPackingApproximationAtLeast
    [DecidableEq Bidder] [DecidableEq Item]
    (sets : Bidder → Finset Item) (weights : Bidder → ℝ)
    (factor : ℝ) (selected : Finset Bidder) : Prop :=
  SetPackingFeasible sets selected ∧
    ∀ optimal, WeightedSetPackingOptimal sets weights optimal →
      factor * weightedSetPackingValue weights optimal ≤
        weightedSetPackingValue weights selected
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WeightedSetPackingApproximationSolverAtLeast`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item] (factor : ℝ)
  (solver : (Bidder → Finset Item) → (Bidder → ℝ) → Finset Bidder) (sets : Bidder → Finset Item) (weights : Bidder → ℝ),
  EconCSLib.Auction.WeightedSetPackingApproximationAtLeast sets weights factor (solver sets weights)
```

Exact Lean library declaration

```
def WeightedSetPackingApproximationSolverAtLeast
    [DecidableEq Bidder] [DecidableEq Item]
    (factor : ℝ)
    (solver :
      (Bidder → Finset Item) → (Bidder → ℝ) → Finset Bidder) : Prop :=
  ∀ sets weights,
    WeightedSetPackingApproximationAtLeast sets weights factor
      (solver sets weights)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WeightedSetPackingDecisionInstance`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
Type u_3 → Type u_4 → Type (max u_3 u_4)
```

Exact Lean library declaration

```
structure WeightedSetPackingDecisionInstance (Bidder Item : Type*) where
  sets : Bidder → Finset Item
  weights : Bidder → ℝ
  threshold : ℝ
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WeightedSetPackingDecisionInstance.mk`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    (Bidder → Finset Item) → (Bidder → ℝ) → ℝ → EconCSLib.Auction.WeightedSetPackingDecisionInstance Bidder Item
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WeightedSetPackingDecisionInstance.sets`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    (self : EconCSLib.Auction.WeightedSetPackingDecisionInstance Bidder Item) → (a : Bidder) → self.1 a
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WeightedSetPackingDecisionInstance.threshold`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} → (self : EconCSLib.Auction.WeightedSetPackingDecisionInstance Bidder Item) → self.3
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WeightedSetPackingDecisionInstance.weights`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_3} →
  {Item : Type u_4} →
    (self : EconCSLib.Auction.WeightedSetPackingDecisionInstance Bidder Item) → (a : Bidder) → self.2 a
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WeightedSetPackingOptimalSolver`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (solver : (Bidder → Finset Item) → (Bidder → ℝ) → Finset Bidder) (sets : Bidder → Finset Item) (weights : Bidder → ℝ),
  EconCSLib.Auction.WeightedSetPackingOptimal sets weights (solver sets weights)
```

Exact Lean library declaration

```
def WeightedSetPackingOptimalSolver
    [DecidableEq Bidder] [DecidableEq Item]
    (solver :
      (Bidder → Finset Item) → (Bidder → ℝ) → Finset Bidder) : Prop :=
  ∀ sets weights, WeightedSetPackingOptimal sets weights (solver sets weights)
```

Recorded source-to-library screening **Verdict:** not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.allocationValue`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : Fintype Bidder] →
      (values : EconCSLib.Auction.CombinatorialReport Bidder Item) →
        (A : EconCSLib.Auction.BundleAllocation Bidder Item) → ∑ i, values i (A i)
```

Exact Lean library declaration

```
noncomputable def allocationValue [Fintype Bidder]
    (values : CombinatorialReport Bidder Item)
    (A : BundleAllocation Bidder Item) : ℝ :=
  ∑ i : Bidder, values i (A i)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.WelfareMaximizingAllocationRule`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : Fintype Bidder]
  (alloc : EconCSLib.Auction.CombinatorialReport Bidder Item → EconCSLib.Auction.BundleAllocation Bidder Item)
  (reports : EconCSLib.Auction.CombinatorialReport Bidder Item) (A : EconCSLib.Auction.BundleAllocation Bidder Item),
  EconCSLib.Auction.allocationValue reports A ≤ EconCSLib.Auction.allocationValue reports (alloc reports)
```

Exact Lean library declaration

```
def WelfareMaximizingAllocationRule [Fintype Bidder]
    (alloc : CombinatorialReport Bidder Item → BundleAllocation Bidder Item) :
    Prop :=
  ∀ reports A, allocationValue reports A ≤ allocationValue reports (alloc reports)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.allocationValueExcept`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : Fintype Bidder] →
      [inst_1 : DecidableEq Bidder] →
        (values : EconCSLib.Auction.CombinatorialReport Bidder Item) →
          (A : EconCSLib.Auction.BundleAllocation Bidder Item) →
            (i : Bidder) → ∑ j ∈ Finset.univ.erase i, values j (A j)
```

Exact Lean library declaration

```
noncomputable def allocationValueExcept [Fintype Bidder] [DecidableEq Bidder]
    (values : CombinatorialReport Bidder Item)
    (A : BundleAllocation Bidder Item) (i : Bidder) : ℝ :=
  (Finset.univ.erase i).sum fun j => values j (A j)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.reportsWithoutBidder`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      (reports : EconCSLib.Auction.CombinatorialReport Bidder Item) →
        (i a : Bidder) → (a_1 : EconCSLib.Auction.Bundle Item) → Function.update reports i (fun x => 0) a a_1
```

Exact Lean library declaration

```
def reportsWithoutBidder [DecidableEq Bidder]
    (reports : CombinatorialReport Bidder Item) (i : Bidder) :
    CombinatorialReport Bidder Item :=
  Function.update reports i fun _ => 0
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.generalizedVickreyAuction`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : Fintype Bidder] →
      [inst_1 : DecidableEq Bidder] →
        (alloc : EconCSLib.Auction.CombinatorialReport Bidder Item → EconCSLib.Auction.BundleAllocation Bidder Item) →
          { allocation := fun reports => alloc reports,
            payment := fun reports i =>
              EconCSLib.Auction.allocationValueExcept reports (alloc (EconCSLib.Auction.reportsWithoutBidder reports i))
                  i -
                EconCSLib.Auction.allocationValueExcept reports (alloc reports) i }
```

Exact Lean library declaration

```
noncomputable def generalizedVickreyAuction
    [Fintype Bidder] [DecidableEq Bidder]
    (alloc : CombinatorialReport Bidder Item → BundleAllocation Bidder Item) :
    CombinatorialAuction Bidder Item where
  allocation reports := alloc reports
  payment reports i :=
    allocationValueExcept reports (alloc (reportsWithoutBidder reports i)) i -
      allocationValueExcept reports (alloc reports) i
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.graphCliqueDecisionToIndependentSetComplementDecision`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Vertex : Type u_3} →
  (problem : EconCSLib.Auction.GraphCliqueDecisionInstance Vertex) →
    { graph := problem.graphᶜ, threshold := problem.threshold }
```

Exact Lean library declaration

```
def graphCliqueDecisionToIndependentSetComplementDecision
    (problem : GraphCliqueDecisionInstance Vertex) :
    GraphIndependentSetDecisionInstance Vertex where
  graph := problem.graphᶜ
  threshold := problem.threshold
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.graphIncidentSets`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Vertex : Type u_3} →
  [inst : Fintype Vertex] →
    [inst_1 : DecidableEq Vertex] →
      (G : SimpleGraph Vertex) → [inst_2 : DecidableRel G.Adj] → (a : Vertex) → G.incidenceFinset a
```

Exact Lean library declaration

```
noncomputable def graphIncidentSets
    [Fintype Vertex] [DecidableEq Vertex]
    (G : SimpleGraph Vertex) [DecidableRel G.Adj] :
    Vertex → Finset (Sym2 Vertex) :=
  fun v => G.incidenceFinset v
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.graphUnitWeights`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(Vertex : Type u_4) → Vertex → 1
```

Exact Lean library declaration

```
def graphUnitWeights (Vertex : Type*) : Vertex → ℝ :=
  fun _ => 1
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.graphIndependentSetDecisionToWeightedSetPackingDecision`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Vertex : Type u_3} →
  [inst : Fintype Vertex] →
    [inst_1 : DecidableEq Vertex] →
      (problem : EconCSLib.Auction.GraphIndependentSetDecisionInstance Vertex) →
        { sets := EconCSLib.Auction.graphIncidentSets problem.graph,
          weights := EconCSLib.Auction.graphUnitWeights Vertex, threshold := problem.threshold }
```

Exact Lean library declaration

```
noncomputable def graphIndependentSetDecisionToWeightedSetPackingDecision
    [Fintype Vertex] [DecidableEq Vertex]
    (problem : GraphIndependentSetDecisionInstance Vertex) :
    WeightedSetPackingDecisionInstance Vertex (Sym2 Vertex) := by
  classical
  exact {
    sets := graphIncidentSets problem.graph
    weights := graphUnitWeights Vertex
    threshold := problem.threshold }
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.graphCliqueDecisionToWeightedSetPackingDecision`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Vertex : Type u_3} →
  [inst : Fintype Vertex] →
    [inst_1 : DecidableEq Vertex] →
      (problem : EconCSLib.Auction.GraphCliqueDecisionInstance Vertex) →
        EconCSLib.Auction.graphIndependentSetDecisionToWeightedSetPackingDecision
          (EconCSLib.Auction.graphCliqueDecisionToIndependentSetComplementDecision problem)
```

Exact Lean library declaration

```
noncomputable def graphCliqueDecisionToWeightedSetPackingDecision
    [Fintype Vertex] [DecidableEq Vertex]
    (problem : GraphCliqueDecisionInstance Vertex) :
    WeightedSetPackingDecisionInstance Vertex (Sym2 Vertex) :=
  graphIndependentSetDecisionToWeightedSetPackingDecision
    (graphCliqueDecisionToIndependentSetComplementDecision problem)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.setPackingSingleMindedBids`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    (sets : Bidder → Finset Item) → (weights : Bidder → ℝ) → (a : Bidder) → { desired := sets a, value := weights a }
```

Exact Lean library declaration

```
def setPackingSingleMindedBids
    (sets : Bidder → Finset Item) (weights : Bidder → ℝ) :
    Bidder → SingleMindedBid Item :=
  fun i => { desired := sets i, value := weights i }
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.setPackingDecisionToSingleMindedWelfareDecision`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    (problem : EconCSLib.Auction.WeightedSetPackingDecisionInstance Bidder Item) →
      { bids := EconCSLib.Auction.setPackingSingleMindedBids problem.sets problem.weights,
        threshold := problem.threshold }
```

Exact Lean library declaration

```
def setPackingDecisionToSingleMindedWelfareDecision
    (problem : WeightedSetPackingDecisionInstance Bidder Item) :
    SingleMindedWelfareDecisionInstance Bidder Item where
  bids := setPackingSingleMindedBids problem.sets problem.weights
  threshold := problem.threshold
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.graphCliqueDecisionToSingleMindedWelfareDecision`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Vertex : Type u_3} →
  [inst : Fintype Vertex] →
    [inst_1 : DecidableEq Vertex] →
      (problem : EconCSLib.Auction.GraphCliqueDecisionInstance Vertex) →
        EconCSLib.Auction.setPackingDecisionToSingleMindedWelfareDecision
          (EconCSLib.Auction.graphCliqueDecisionToWeightedSetPackingDecision problem)
```

Exact Lean library declaration

```
noncomputable def graphCliqueDecisionToSingleMindedWelfareDecision
    [Fintype Vertex] [DecidableEq Vertex]
    (problem : GraphCliqueDecisionInstance Vertex) :
    SingleMindedWelfareDecisionInstance Vertex (Sym2 Vertex) :=
  setPackingDecisionToSingleMindedWelfareDecision
    (graphCliqueDecisionToWeightedSetPackingDecision problem)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.setPackingSolverOfSingleMindedSolver`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    (solver : (Bidder → EconCSLib.Auction.SingleMindedBid Item) → Finset Bidder) →
      (sets : Bidder → Finset Item) →
        (weights : Bidder → ℝ) → solver (EconCSLib.Auction.setPackingSingleMindedBids sets weights)
```

Exact Lean library declaration

```
def setPackingSolverOfSingleMindedSolver
    (solver : (Bidder → SingleMindedBid Item) → Finset Bidder)
    (sets : Bidder → Finset Item) (weights : Bidder → ℝ) : Finset Bidder :=
  solver (setPackingSingleMindedBids sets weights)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedAverageTieKey`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [DecidableEq Item] →
      [LinearOrder Bidder] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (i : Bidder) → toLex (OrderDual.toDual (bids i).averageAmountPerGood, i)
```

Exact Lean library declaration

```
noncomputable def singleMindedAverageTieKey
    [DecidableEq Item] [LinearOrder Bidder]
    (bids : Bidder → SingleMindedBid Item) (i : Bidder) :
    (ℝᵒᵈ ×ₗ Bidder) :=
  toLex (OrderDual.toDual (bids i).averageAmountPerGood, i)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedAverageTieRel`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] [inst_1 : LinearOrder Bidder]
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) (a a_1 : Bidder),
  (EconCSLib.Auction.singleMindedAverageTieKey bids ⁻¹'o fun x1 x2 => x1 ≤ x2) a a_1
```

Exact Lean library declaration

```
noncomputable def singleMindedAverageTieRel
    [DecidableEq Item] [LinearOrder Bidder]
    (bids : Bidder → SingleMindedBid Item) : Bidder → Bidder → Prop :=
  Order.Preimage (singleMindedAverageTieKey bids) (· ≤ ·)
```

Recorded source-to-library screening **Verdict:** not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedAverageTieRelAntisymm`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] [inst_1 : LinearOrder Bidder]
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item),
  Std.Antisymm (EconCSLib.Auction.singleMindedAverageTieRel bids)
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedAverageTieRelDecidable`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Item] →
      [inst_1 : LinearOrder Bidder] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) → (a b : Bidder) → id inferInstance a b
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedAverageTieRelTotal`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] [inst_1 : LinearOrder Bidder]
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item), Std.Total (EconCSLib.Auction.singleMindedAverageTieRel bids)
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedAverageTieRelTrans`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] [inst_1 : LinearOrder Bidder]
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item),
  IsTrans Bidder (EconCSLib.Auction.singleMindedAverageTieRel bids)
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedAverageOrderOf`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : Fintype Bidder] →
      [inst_1 : DecidableEq Item] →
        [inst_2 : LinearOrder Bidder] →
          (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
            Finset.univ.sort (EconCSLib.Auction.singleMindedAverageTieRel bids)
```

Exact Lean library declaration

```
noncomputable def singleMindedAverageOrderOf
    [Fintype Bidder] [DecidableEq Item] [LinearOrder Bidder]
    (bids : Bidder → SingleMindedBid Item) : List Bidder :=
  (Finset.univ : Finset Bidder).sort (singleMindedAverageTieRel bids)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyConflictingAccepted`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [DecidableEq Bidder] →
      [inst : DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (accepted : Finset Bidder) →
            (i : Bidder) → {j ∈ accepted | Finset.Nonempty ((bids i).desired ∩ (bids j).desired)}
```

Exact Lean library declaration

```
def singleMindedGreedyConflictingAccepted [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (accepted : Finset Bidder) (i : Bidder) : Finset Bidder :=
  accepted.filter fun j => ((bids i).desired ∩ (bids j).desired).Nonempty
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyStep`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (accepted : Finset Bidder) →
            (i : Bidder) →
              if (EconCSLib.Auction.singleMindedGreedyConflictingAccepted bids accepted i).Nonempty then accepted
              else insert i accepted
```

Exact Lean library declaration

```
def singleMindedGreedyStep [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (accepted : Finset Bidder) (i : Bidder) : Finset Bidder :=
  if (singleMindedGreedyConflictingAccepted bids accepted i).Nonempty then
    accepted
  else
    insert i accepted
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyAcceptedFromState`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (accepted : Finset Bidder) →
            (order : List Bidder) → List.foldl (EconCSLib.Auction.singleMindedGreedyStep bids) accepted order
```

Exact Lean library declaration

```
def singleMindedGreedyAcceptedFromState [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (accepted : Finset Bidder) (order : List Bidder) :
    Finset Bidder :=
  order.foldl (singleMindedGreedyStep bids) accepted
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyAcceptedFromOrder`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (order : List Bidder) → EconCSLib.Auction.singleMindedGreedyAcceptedFromState bids ∅ order
```

Exact Lean library declaration

```
def singleMindedGreedyAcceptedFromOrder [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item) (order : List Bidder) :
    Finset Bidder :=
  singleMindedGreedyAcceptedFromState bids ∅ order
```

Recorded source-to-library screening **Verdict:** not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyDeniedBecauseOfAtStateBool`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (acceptedBefore : Finset Bidder) →
            (j i : Bidder) →
              decide (j ∈ acceptedBefore) && decide (i ∉ acceptedBefore) &&
                  decide (Finset.Nonempty ((bids i).desired ∩ (bids j).desired)) &&
                decide (EconCSLib.Auction.singleMindedGreedyConflictingAccepted bids (acceptedBefore.erase j) i = ∅)
```

Exact Lean library declaration

```
def singleMindedGreedyDeniedBecauseOfAtStateBool
    [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (acceptedBefore : Finset Bidder) (j i : Bidder) : Bool :=
  decide (j ∈ acceptedBefore) &&
    decide (i ∉ acceptedBefore) &&
      decide (((bids i).desired ∩ (bids j).desired).Nonempty) &&
        decide
          ((singleMindedGreedyConflictingAccepted
              bids (acceptedBefore.erase j) i) = ∅)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyFirstDeniedBecauseOfFromState`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (acceptedBeforeSuffix : Finset Bidder) →
            (suffix : List Bidder) →
              (j : Bidder) →
                List.brecOn (motive := fun suffix => Finset Bidder → Option Bidder) suffix
                  (fun suffix f acceptedBeforeSuffix =>
                    (match (motive :=
                        (suffix : List Bidder) →
                          List.below (motive := fun suffix => Finset Bidder → Option Bidder) suffix → Option Bidder)
                        suffix with
                      | [] => fun x => none
                      | i :: rest => fun x =>
                        match
                          EconCSLib.Auction.singleMindedGreedyDeniedBecauseOfAtStateBool bids acceptedBeforeSuffix j
                            i with
                        | true => some i
                        | false => x.1 (EconCSLib.Auction.singleMindedGreedyStep bids acceptedBeforeSuffix i))
                      f)
                  acceptedBeforeSuffix
```

Exact Lean library declaration

```
def singleMindedGreedyFirstDeniedBecauseOfFromState
    [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (acceptedBeforeSuffix : Finset Bidder)
    (suffix : List Bidder) (j : Bidder) : Option Bidder :=
  match suffix with
  | [] => none
  | i :: rest =>
      match singleMindedGreedyDeniedBecauseOfAtStateBool
          bids acceptedBeforeSuffix j i with
      | true => some i
      | false =>
          singleMindedGreedyFirstDeniedBecauseOfFromState bids
            (singleMindedGreedyStep bids acceptedBeforeSuffix i) rest j
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyNextDeniedFromStateInOrder`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (acceptedBeforeOrder : Finset Bidder) →
            (order : List Bidder) →
              (j : Bidder) →
                List.brecOn (motive := fun order => Finset Bidder → Option Bidder) order
                  (fun order f acceptedBeforeOrder =>
                    (match (motive :=
                        (order : List Bidder) →
                          List.below (motive := fun order => Finset Bidder → Option Bidder) order → Option Bidder)
                        order with
                      | [] => fun x => none
                      | i :: rest => fun x =>
                        if i = j then
                          EconCSLib.Auction.singleMindedGreedyFirstDeniedBecauseOfFromState bids
                            (EconCSLib.Auction.singleMindedGreedyStep bids acceptedBeforeOrder i) rest j
                        else x.1 (EconCSLib.Auction.singleMindedGreedyStep bids acceptedBeforeOrder i))
                      f)
                  acceptedBeforeOrder
```

Exact Lean library declaration

```
def singleMindedGreedyNextDeniedFromStateInOrder
    [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (acceptedBeforeOrder : Finset Bidder)
    (order : List Bidder) (j : Bidder) : Option Bidder :=
  match order with
  | [] => none
  | i :: rest =>
      if i = j then
        singleMindedGreedyFirstDeniedBecauseOfFromState bids
          (singleMindedGreedyStep bids acceptedBeforeOrder i) rest j
      else
        singleMindedGreedyNextDeniedFromStateInOrder bids
          (singleMindedGreedyStep bids acceptedBeforeOrder i) rest j
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyNextDeniedFromOrder`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (order : List Bidder) →
            (j : Bidder) → EconCSLib.Auction.singleMindedGreedyNextDeniedFromStateInOrder bids ∅ order j
```

Exact Lean library declaration

```
def singleMindedGreedyNextDeniedFromOrder
    [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (order : List Bidder) (j : Bidder) : Option Bidder :=
  singleMindedGreedyNextDeniedFromStateInOrder bids ∅ order j
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyPaymentFromNextDenied`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (accepted : Finset Bidder) →
            (nextDenied : Bidder → Option Bidder) →
              (j : Bidder) →
                if j ∈ accepted then
                  match nextDenied j with
                  | none => 0
                  | some n => (bids j).bundleSize * (bids n).averageAmountPerGood
                else 0
```

Exact Lean library declaration

```
noncomputable def singleMindedGreedyPaymentFromNextDenied
    [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (accepted : Finset Bidder)
    (nextDenied : Bidder → Option Bidder) (j : Bidder) : ℝ :=
  if j ∈ accepted then
    match nextDenied j with
    | none => 0
    | some n => (bids j).bundleSize * (bids n).averageAmountPerGood
  else
    0
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyPaymentFromOrder`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) →
          (order : List Bidder) →
            (j : Bidder) →
              EconCSLib.Auction.singleMindedGreedyPaymentFromNextDenied bids
                (EconCSLib.Auction.singleMindedGreedyAcceptedFromOrder bids order)
                (EconCSLib.Auction.singleMindedGreedyNextDeniedFromOrder bids order) j
```

Exact Lean library declaration

```
noncomputable def singleMindedGreedyPaymentFromOrder
    [DecidableEq Bidder] [DecidableEq Item]
    (bids : Bidder → SingleMindedBid Item)
    (order : List Bidder) (j : Bidder) : ℝ :=
  singleMindedGreedyPaymentFromNextDenied bids
    (singleMindedGreedyAcceptedFromOrder bids order)
    (singleMindedGreedyNextDeniedFromOrder bids order) j
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Auction.singleMindedGreedyAcceptedMechanismFromOrderOf`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Bidder : Type u_1} →
  {Item : Type u_2} →
    [inst : DecidableEq Bidder] →
      [inst_1 : DecidableEq Item] →
        (orderOf : (Bidder → EconCSLib.Auction.SingleMindedBid Item) → List Bidder) →
          { accepted := fun bids => EconCSLib.Auction.singleMindedGreedyAcceptedFromOrder bids (orderOf bids),
            payment := fun bids j => EconCSLib.Auction.singleMindedGreedyPaymentFromOrder bids (orderOf bids) j }
```

Exact Lean library declaration

```
noncomputable def singleMindedGreedyAcceptedMechanismFromOrderOf
    [DecidableEq Bidder] [DecidableEq Item]
    (orderOf : (Bidder → SingleMindedBid Item) → List Bidder) :
    SingleMindedAcceptedMechanism Bidder Item where
  accepted bids := singleMindedGreedyAcceptedFromOrder bids (orderOf bids)
  payment bids j := singleMindedGreedyPaymentFromOrder bids (orderOf bids) j
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.ComplexityClassModel`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
Type u → Type u
```

Exact Lean library declaration

```
structure ComplexityClassModel (Instance : Type u) where
  NP : Set (DecisionProblem Instance)
  ZPP : Set (DecisionProblem Instance)
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.DecisionProblem`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(Instance : Type u) → Instance → Prop
```

Exact Lean library declaration

```
abbrev DecisionProblem (Instance : Type u) := Instance → Prop
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.ComplexityClassModel.NP`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Instance : Type u} (self : EconCSLib.Complexity.ComplexityClassModel Instance)
  (a : EconCSLib.Complexity.DecisionProblem Instance), self.1 a
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.ComplexityClassModel.ZPP`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Instance : Type u} (self : EconCSLib.Complexity.ComplexityClassModel Instance)
  (a : EconCSLib.Complexity.DecisionProblem Instance), self.2 a
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.ComplexityClassModel.npEqZPP`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Instance : Type u} (M : EconCSLib.Complexity.ComplexityClassModel Instance), M.NP = M.ZPP
```

Exact Lean library declaration

```
def npEqZPP (M : ComplexityClassModel Instance) : Prop :=
  M.NP = M.ZPP
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.RandomizedComplexityClassModel`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
Type u → Type u
```

Exact Lean library declaration

```
structure RandomizedComplexityClassModel (Instance : Type u) extends
    ComplexityClassModel Instance where
  P : Set (DecisionProblem Instance)
  RP : Set (DecisionProblem Instance)
  coRP : Set (DecisionProblem Instance)
  coNP : Set (DecisionProblem Instance)
  P_subset_ZPP : P ⊆ ZPP
  ZPP_subset_NP : ZPP ⊆ NP
  RP_subset_NP : RP ⊆ NP
  ZPP_eq_RP_inter_coRP : ZPP = RP ∩ coRP
  coRP_eq_complement_RP : coRP = ComplementClass RP
  coNP_eq_complement_NP : coNP = ComplementClass NP
  ZPP_complement_closed : ComplementClass ZPP = ZPP
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.RandomizedComplexityClassModel.RP`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Instance : Type u} (self : EconCSLib.Complexity.RandomizedComplexityClassModel Instance)
  (a : EconCSLib.Complexity.DecisionProblem Instance), self.3 a
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.RandomizedComplexityClassModel.coNP`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Instance : Type u} (self : EconCSLib.Complexity.RandomizedComplexityClassModel Instance)
  (a : EconCSLib.Complexity.DecisionProblem Instance), self.5 a
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.RandomizedComplexityClassModel.coRP`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Instance : Type u} (self : EconCSLib.Complexity.RandomizedComplexityClassModel Instance)
  (a : EconCSLib.Complexity.DecisionProblem Instance), self.4 a
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.Complexity.RandomizedComplexityClassModel.toComplexityClassModel`

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Instance : Type u} → (self : EconCSLib.Complexity.RandomizedComplexityClassModel Instance) → self.1
```

Exact Lean library declaration

```
library declaration is not registered or discoverable
```

Recorded source-to-library screening **Verdict:** `not recorded`

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 226-252

Verbatim source input

```
Since we assume the Independent Value Model and Quasi-Linear utilities, fairly
standard assumptions in the field, the utility, for a bidder of type t, of bundle s ⊆ G
and payment x is:
u = t(s) − x.

(1)

Definition 3.1. A (direct) mechanism for combinatorial auctions consists of
−an allocation algorithm f that picks, for each vector D (D is a vector of declared
types), an allocation f (D),
−a payment scheme p that determines, for each vector D a payment vector p(D):
pi (D) is paid by bidder i to the auctioneer.
Let us denote the bundle obtained by i as:
gi (D) = f (D)−1 (i).

(2)

Notation. In general, gi depends on the allocation algorithm f , but when f is
clear from the context, we shall abuse the notation and treat gi as a direct function
of the bid vector, D. Equation (1) implies that if bidder i has (true) type t, his utility


[form-feed in source extraction]
582

D. LEHMANN ET AL.

from the mechanism is:
u i = t(gi (D)) − pi (D),
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} (M : EconCSLib.Auction.CombinatorialAuction Bidder Item)
  (values reports : EconCSLib.Auction.CombinatorialReport Bidder Item) (i : Bidder),
  LOS02CombinatorialAuctions.PaperInterface.utility M values reports i =
    values i (M.allocation reports i) - M.payment reports i
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.utility_formula
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 258-272

Verbatim source input

```
The game-theoretic solution concept used throughout this article is that of a weakly
dominant strategy, that is, a strategy that is as good as any other for a given player,
no matter what other players do. This is in contrast with the weaker and more
common notion of Nash equilibrium. The particular property we would like to
ensure for our mechanism is that the dominant strategy for each player is to bid
his true valuation; in other words, no bidder can be better off by lying, no matter
how other bidders behave. A mechanism is truthful if no bidder can be better off by
lying, even if other bidders lie. This is a very strong requirement, making for a very
sturdy mechanism.
Definition 3.2. A mechanism h f, pi is truthful if and only if for every i ∈ P,
t ∈ 2 and any vector D of declarations, if D 0 is the vector obtained from D by
replacing the ith coordinate di by t, then: t(gi (D 0 )) − pi (D 0 ) ≥ t(gi (D)) − pi (D).
In the definition above, t is the true type of bidder i and D is a vector of declared
types. The term, t(gi (D)), represents the true satisfaction i receives from the allocation resulting from declarations D and
↪  t(gi (D 0 )) represents his true satisfaction
from the allocation that would have been obtained had i been truthful.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder]
  (M : EconCSLib.Auction.CombinatorialAuction Bidder Item)
  (admissible : EconCSLib.Auction.CombinatorialReport Bidder Item → Prop),
  LOS02CombinatorialAuctions.PaperInterface.truthfulOn M admissible ↔
    ∀ (values : EconCSLib.Auction.CombinatorialReport Bidder Item),
      admissible values →
        ∀ (i : Bidder) (report : EconCSLib.Auction.Bundle Item → ℝ),
          LOS02CombinatorialAuctions.PaperInterface.utility M values (Function.update values i report) i ≤
            LOS02CombinatorialAuctions.PaperInterface.utility M values values i
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.truthfulOn_iff
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 282-339

Verbatim source input

```
In a GVA, the allocation chosen maximizes the sum of the declared valuations
of the bidders, each bidder receives a monetary amount that equals the sum of the
declared valuations of all other bidders, and pays the auctioneer the sum of such
valuations that would have been obtained if he had not participated in the auction.
A way to describe such an auction, in which i does not participate, is to consider
the auction in which bidder i declares a zero valuation for all possible bundles. A
bidder with zero valuation for all bundles has no influence on the outcome.
Formally, given a vector D of declarations, the generalized Vickrey auction
defines the allocation and payment policies as follows (notice that a −1 (i) is the
bundle allocated to i by allocation a, and that gi is defined in Eq. (2)):
f (D) = argmaxa∈O

n
X
i=1

di (a −1 (i)),

(4)


[form-feed in source extraction]
Approximately Efficient Combinatorial Auctions
n
X

p j (D) = −

n
X

di (gi (D)) +

i=1,i6= j

583

di (gi (Z )),

(5)

i=1,i6= j

where Z i = Di for any i 6= j and Z j (s) = 0 for any bundle s ⊆ G. Since
d j (g j (Z )) = 0, we may as well have written:
p j (D) = −

n
X

di (gi (D)) +

i=1,i6= j

n
X

di (gi (Z )).

(6)
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : Fintype Bidder] [inst_1 : DecidableEq Bidder]
  (alloc : EconCSLib.Auction.CombinatorialReport Bidder Item → EconCSLib.Auction.BundleAllocation Bidder Item)
  (reports : EconCSLib.Auction.CombinatorialReport Bidder Item) (i : Bidder),
  (LOS02CombinatorialAuctions.PaperInterface.generalizedVickreyAuction alloc).allocation reports = alloc reports ∧
    (LOS02CombinatorialAuctions.PaperInterface.generalizedVickreyAuction alloc).payment reports i =
      EconCSLib.Auction.allocationValueExcept reports (alloc (EconCSLib.Auction.reportsWithoutBidder reports i)) i -
        EconCSLib.Auction.allocationValueExcept reports (alloc reports) i
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.generalizedVickreyAuction_allocation_payment
```

Recorded source-to-Spec screening **Verdict:** `not recorded`

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 449-465

Verbatim source input

```
We shall assume that bidders are single-minded and care only about one specific
(bidder-dependent) set of goods. If they do not get this set they value the outcome
at the lowest possible value 0. In other words, our bidders are restricted to one
single bid.
Definition 5.1. Bidder i is single-minded if and only if there is a set s ⊂ G of
goods and a value v ∈ R+ such that its type t can be described as: t(s 0 ) = v if s ⊆ s 0
and t(s 0 ) = 0 otherwise.
Note that single-minded bidders are not one parameter agents in the sense
of Archer and Tardos [2001], since they have the freedom of deciding which bundle
they bid for on top on the amount they are willing to pay. In Mu'alem and Nisan
[2002], truthfulness is shown to be attainable for allocation algorithms providing
better approximations if agents are assumed to be single-minded and one parameter. We shall denote by hs, vi the type just
↪  described. Note that a single-minded
bidder enjoys free disposal. We shall assume, in most of this article, that all bidders
are single-minded, that is, there are sets of goods si and nonnegative real numbers
v i such that bidder i is of type hsi , v i i. We shall denote by 6 the set of all singleminded types. The size of the set 6,
↪  contrary to the size of 2, is singly exponential
in k. A string of polynomial size will be enough to code the declarations of the
bidders: it will describe a set of goods and a value. In this setting, we identify bids
```

Expanded PaperInterface specification

```
∀ (Bidder : Type u_1) (Item : Type u_2)
  (M : LOS02CombinatorialAuctions.PaperInterface.singleMindedAcceptedMechanism Bidder Item), M = M
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.singleMindedAcceptedMechanism_fields
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 449-465

Verbatim source input

```
We shall assume that bidders are single-minded and care only about one specific
(bidder-dependent) set of goods. If they do not get this set they value the outcome
at the lowest possible value 0. In other words, our bidders are restricted to one
single bid.
Definition 5.1. Bidder i is single-minded if and only if there is a set s ⊂ G of
goods and a value v ∈ R+ such that its type t can be described as: t(s 0 ) = v if s ⊆ s 0
and t(s 0 ) = 0 otherwise.
Note that single-minded bidders are not one parameter agents in the sense
of Archer and Tardos [2001], since they have the freedom of deciding which bundle
they bid for on top on the amount they are willing to pay. In Mu'alem and Nisan
[2002], truthfulness is shown to be attainable for allocation algorithms providing
better approximations if agents are assumed to be single-minded and one parameter. We shall denote by hs, vi the type just
↪  described. Note that a single-minded
bidder enjoys free disposal. We shall assume, in most of this article, that all bidders
are single-minded, that is, there are sets of goods si and nonnegative real numbers
v i such that bidder i is of type hsi , v i i. We shall denote by 6 the set of all singleminded types. The size of the set 6,
↪  contrary to the size of 2, is singly exponential
in k. A string of polynomial size will be enough to code the declarations of the
bidders: it will describe a set of goods and a value. In this setting, we identify bids
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item)
  (admissible : (Bidder → EconCSLib.Auction.SingleMindedBid Item) → Prop),
  LOS02CombinatorialAuctions.PaperInterface.singleMindedTruthfulOn M admissible ↔
    ∀ (values : Bidder → EconCSLib.Auction.SingleMindedBid Item),
      admissible values →
        ∀ (i : Bidder) (report : EconCSLib.Auction.SingleMindedBid Item),
          admissible (Function.update values i report) →
            M.utility values (Function.update values i report) i ≤ M.utility values values i
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.singleMindedTruthfulOn_iff
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 449-465

Verbatim source input

```
We shall assume that bidders are single-minded and care only about one specific
(bidder-dependent) set of goods. If they do not get this set they value the outcome
at the lowest possible value 0. In other words, our bidders are restricted to one
single bid.
Definition 5.1. Bidder i is single-minded if and only if there is a set s ⊂ G of
goods and a value v ∈ R+ such that its type t can be described as: t(s 0 ) = v if s ⊆ s 0
and t(s 0 ) = 0 otherwise.
Note that single-minded bidders are not one parameter agents in the sense
of Archer and Tardos [2001], since they have the freedom of deciding which bundle
they bid for on top on the amount they are willing to pay. In Mu'alem and Nisan
[2002], truthfulness is shown to be attainable for allocation algorithms providing
better approximations if agents are assumed to be single-minded and one parameter. We shall denote by hs, vi the type just
↪  described. Note that a single-minded
bidder enjoys free disposal. We shall assume, in most of this article, that all bidders
are single-minded, that is, there are sets of goods si and nonnegative real numbers
v i such that bidder i is of type hsi , v i i. We shall denote by 6 the set of all singleminded types. The size of the set 6,
↪  contrary to the size of 2, is singly exponential
in k. A string of polynomial size will be enough to code the declarations of the
bidders: it will describe a set of goods and a value. In this setting, we identify bids
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Item]
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item),
  LOS02CombinatorialAuctions.PaperInterface.nonnegativeNonemptySingleMindedProfile bids ↔
    ∀ (i : Bidder), Finset.Nonempty (bids i).desired ∧ 0 ≤ (bids i).value
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.nonnegativeNonemptySingleMindedProfile_iff
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 505-523

Verbatim source input

```
In a GVA, the allocation
Pn is the one defined in Eq. (4). Computing this allocation
di (a) over all a's in the set O that is of exponential size.
requires optimizing i=1
One may suspect that this an infeasible task. Indeed, the problem of finding the
allocation of Eq. (4) has been shown to be NP-hard in Rothkopf et al. [1998]. We
remark that the restriction to single-minded bidders does nothing to alleviate the
problem. Not only is it infeasible to solve exactly the optimization problem, but
it turns out to be also infeasible to guarantee any nontrivial approximation to the
optimal allocation. This follows easily from a celebrated result of Håstad [1999]
and was, independently, noticed by Sandholm to whom priority is due: final version
in Sandholm [2002].
THEOREM 6.1. Let a single-minded type di = hsi , v i i, si ⊆ G, v i ∈ R+ be given
for each bidder i ∈ P. Let
Pn|G| = k and |P| = n. The problem of finding an allodi (a) is NP-hard in k + n. Moreover, the existence
cation a that maximizes i=1
of a polynomial time algorithm guaranteed to find an allocation whose value is at
least k −1/2+² times the value of the optimal solution would imply that NP = ZPP.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} [inst : DecidableEq Bidder] (weights : Bidder → ℝ) (selected : Finset Bidder),
  LOS02CombinatorialAuctions.PaperInterface.weightedSetPackingValue weights selected = ∑ i ∈ selected, weights i
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.weightedSetPackingValue_formula
```

Recorded source-to-Spec screening **Verdict:** `not recorded`

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 501-523

Verbatim source input

```
6. Infeasibility of the GVA
Let us now assume that all bidders are single-minded, that is, the set of all possible
types is now 6. It follows easily from Proposition 4.2 that, in a GVA, a singleminded bidder of type hs, vi never pays more than
↪  v and pays nothing if he is not
allocated the whole set s.
In a GVA, the allocation
Pn is the one defined in Eq. (4). Computing this allocation
di (a) over all a's in the set O that is of exponential size.
requires optimizing i=1
One may suspect that this an infeasible task. Indeed, the problem of finding the
allocation of Eq. (4) has been shown to be NP-hard in Rothkopf et al. [1998]. We
remark that the restriction to single-minded bidders does nothing to alleviate the
problem. Not only is it infeasible to solve exactly the optimization problem, but
it turns out to be also infeasible to guarantee any nontrivial approximation to the
optimal allocation. This follows easily from a celebrated result of Håstad [1999]
and was, independently, noticed by Sandholm to whom priority is due: final version
in Sandholm [2002].
THEOREM 6.1. Let a single-minded type di = hsi , v i i, si ⊆ G, v i ∈ R+ be given
for each bidder i ∈ P. Let
Pn|G| = k and |P| = n. The problem of finding an allodi (a) is NP-hard in k + n. Moreover, the existence
cation a that maximizes i=1
of a polynomial time algorithm guaranteed to find an allocation whose value is at
least k −1/2+² times the value of the optimal solution would imply that NP = ZPP.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} (sets : Bidder → Finset Item) (weights : Bidder → ℝ) (i : Bidder),
  LOS02CombinatorialAuctions.PaperInterface.setPackingSingleMindedBids sets weights i =
    { desired := sets i, value := weights i }
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.setPackingSingleMindedBids_formula
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 630-636

Verbatim source input

```
a
.
Definition 7.1. The average amount per good of a bid b = hs, ai is |s|

Sorting the list L by descending average amount per good is a very reasonable idea. But many other possibilities may be
↪  considered. Sorting L by descending amounts for example, or, more generally sorting L by a criterion of the form
a/|s|l for some number l, l ≥ 0, possibly depending on k. All such criteria satisfy
bid-monotonicity.
```

Expanded PaperInterface specification

```
∀ {Item : Type u_1} [inst : DecidableEq Item] (b : EconCSLib.Auction.SingleMindedBid Item),
  LOS02CombinatorialAuctions.PaperInterface.averageAmountPerGood b = b.value / b.bundleSize
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.averageAmountPerGood_formula
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 590-624

Verbatim source input

```
The algorithms we consider execute in two phases.
—In the first phase, the bids are sorted by some criterion. The algorithms of the
family are distinguished by the different criteria they use. Since there are n bids,
this phase takes time of the order of n log n. We assume a criterion, that is, a norm
is defined and the bids are sorted in decreasing order following this norm. Since
we shall have, in Theorem 10.2, to compare the sorted lists of bids of slightly
different auctions, we also assume a consistent treatment of ties, that is, bids
with equal norms. Formally, we shall assume that no two different bids have the
same norm, that is, there are no ties.
—In the second phase, a greedy algorithm generates an allocation. Let L be the
list of sorted bids obtained in the first phase. The first bid of L, say b = hs, ai is
granted, that is, the set s will be allocated to b and then the algorithm examines
each bid of L, in order, and grants it if it does not conflict with any of the bids
previously granted. If it does, it denies (i.e., does not grant) the bid. This phase
requires time linear in n.
The use of such a greedy scheme is very straightforward and speedy. We shall
now discuss its efficiency: how efficient is the allocation generated? The efficiency
of the allocation generated depends obviously both on the criterion used in the first
phase and on the types of the bidders, or on the distribution with which the bidders
are generated. It is clear that, to obtain allocations close to efficiency, one should use
a norm that pushes bids that have a good chance to be part of an efficient allocation
toward the beginning of the list L. The amount of a bid is a good criterion in this respect: we want bids with higher amounts to
↪  have a larger norm than bids with lower
amounts, at least when the bids are for the same set of goods. Similarly, leaving the
amount of a bid unchanged but making its bundle a smaller set (inclusion-wise),
should also increase the norm. We shall require that changing s to s 0 with s 0 ⊂ s
or changing v to v 0 with v 0 > v increase the norm of a bid. Let us call this property
bid-monotonicity. This is the only requirement we shall make. Many criteria
satisfy it.
In real-life situations, one can typically find a suitable natural norm related to the
economic parameters of the bundle that measures the a-priori attractiveness of the
bid (for the auctioneer). In the FCC auction, goods (licenses) are characterized by
the population they cover. The (inverse of the) sum of those populations is a good
indicator. In the abstract, if we know nothing concrete about the goods, our best bet
is to use the size of the set of goods mentioned in a bid. We shall look in particular
at the average-amount-per-good measure.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : Fintype Bidder] [inst_1 : DecidableEq Item]
  [inst_2 : LinearOrder Bidder] (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item),
  (LOS02CombinatorialAuctions.PaperInterface.averageOrderOf bids).Nodup ∧
    (∀ (i : Bidder), i ∈ LOS02CombinatorialAuctions.PaperInterface.averageOrderOf bids) ∧
      EconCSLib.Auction.SingleMindedAverageAmountDescending bids
        (LOS02CombinatorialAuctions.PaperInterface.averageOrderOf bids)
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.averageOrderOf_rule
```

Recorded source-to-Spec screening **Verdict:** `not recorded`

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 590-604

Verbatim source input

```
The algorithms we consider execute in two phases.
—In the first phase, the bids are sorted by some criterion. The algorithms of the
family are distinguished by the different criteria they use. Since there are n bids,
this phase takes time of the order of n log n. We assume a criterion, that is, a norm
is defined and the bids are sorted in decreasing order following this norm. Since
we shall have, in Theorem 10.2, to compare the sorted lists of bids of slightly
different auctions, we also assume a consistent treatment of ties, that is, bids
with equal norms. Formally, we shall assume that no two different bids have the
same norm, that is, there are no ties.
—In the second phase, a greedy algorithm generates an allocation. Let L be the
list of sorted bids obtained in the first phase. The first bid of L, say b = hs, ai is
granted, that is, the set s will be allocated to b and then the algorithm examines
each bid of L, in order, and grants it if it does not conflict with any of the bids
previously granted. If it does, it denies (i.e., does not grant) the bid. This phase
requires time linear in n.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) (order : List Bidder),
  LOS02CombinatorialAuctions.PaperInterface.greedyAcceptedFromOrder bids order =
    List.foldl (EconCSLib.Auction.singleMindedGreedyStep bids) ∅ order
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.greedyAcceptedFromOrder_formula
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 590-604

Verbatim source input

```
The algorithms we consider execute in two phases.
—In the first phase, the bids are sorted by some criterion. The algorithms of the
family are distinguished by the different criteria they use. Since there are n bids,
this phase takes time of the order of n log n. We assume a criterion, that is, a norm
is defined and the bids are sorted in decreasing order following this norm. Since
we shall have, in Theorem 10.2, to compare the sorted lists of bids of slightly
different auctions, we also assume a consistent treatment of ties, that is, bids
with equal norms. Formally, we shall assume that no two different bids have the
same norm, that is, there are no ties.
—In the second phase, a greedy algorithm generates an allocation. Let L be the
list of sorted bids obtained in the first phase. The first bid of L, say b = hs, ai is
granted, that is, the set s will be allocated to b and then the algorithm examines
each bid of L, in order, and grants it if it does not conflict with any of the bids
previously granted. If it does, it denies (i.e., does not grant) the bid. This phase
requires time linear in n.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : Fintype Bidder] [inst_1 : DecidableEq Bidder]
  [inst_2 : DecidableEq Item] [inst_3 : LinearOrder Bidder] (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item),
  LOS02CombinatorialAuctions.PaperInterface.averageGreedyAcceptedSet bids =
    LOS02CombinatorialAuctions.PaperInterface.greedyAcceptedFromOrder bids
      (LOS02CombinatorialAuctions.PaperInterface.averageOrderOf bids)
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.averageGreedyAcceptedSet_formula
```

Recorded source-to-Spec screening **Verdict:** `not recorded`

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: cited publication, lines 969-993

Verbatim source input

```
10. A Truthful Mechanism with Greedy Allocation
We shall now describe the payment mechanism that we propose to be used in
conjunction with the greedy allocation of Section 7. The description of the payments
is tightly linked with that of the greedy algorithm. The computation of the payment
is performed in parallel with the execution of the greedy algorithm and takes time
linear in the number of bidders for each payment. On the whole, computing the
allocation and the payments takes time at most quadratic in the number of bids.
We assume that the criterion used is average amount per good, the adaptation
to most other suitable greedy allocations is obvious. Informally, a bidder pays, per
good, the average price proposed by the first bid in the list L that is denied because
of this bid. Consider a bid j in L. Let c( j) be the average amount per good of j. We
shall denote by n( j) the first bid following j (bids are sorted in decreasing order,
that is, c( j) ≥ c(n( j))) that has been denied but would have been granted were it
not for the presence of j. Assume that such a bid exists. Notice that such a bid
necessarily conflicts with j, and therefore:
n( j) = min{i | j < i, s( j) ∩ s(i) 6= ∅, ∀l, l < i, l 6= j, l granted ⇒ s(l) ∩ s(i) = ∅}.
Definition 10.1 Greedy Payment Scheme.
the first phase.

Let L be the sorted list obtained in

– j pays zero if his bid is denied or if there is no bid n( j),
–if there is an n( j) and j's bid hs, vi is granted, he pays |s| × c(n( j)).
We may now state the main result of this article.
THEOREM 10.2. The mechanism composed of the greedy allocation and payment schemes is truthful for single-minded bidders.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : Fintype Bidder] [inst_1 : DecidableEq Bidder]
  [inst_2 : DecidableEq Item] [inst_3 : LinearOrder Bidder] (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item)
  (j : Bidder),
  LOS02CombinatorialAuctions.PaperInterface.averageGreedyPayment bids j =
    if j ∈ LOS02CombinatorialAuctions.PaperInterface.averageGreedyAcceptedSet bids then
      match
        LOS02CombinatorialAuctions.paper_greedy_next_denied_from_order bids
          (LOS02CombinatorialAuctions.PaperInterface.averageOrderOf bids) j with
      | none => 0
      | some n => (bids j).bundleSize * (bids n).averageAmountPerGood
    else 0
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.averageGreedyPayment_formula
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 14

Source locator: cited publication, lines 343-385

Verbatim source input

```
A proof of the truthfulness of the Clarke–Groves–Vickrey mechanism may be
found, for example, in Mas-Colell et al. [1995, Proposition 23.C.4]. We include the
proof here only to stress how easy it is.
THEOREM 4.1. The generalized Vickrey auction is a truthful mechanism.
PROOF. Assume j ∈ P, t ∈ 2, D is a vector of declarations, and Di0 = Di for
any i 6= j and D 0j = t. By Eq. (4),
n
X

di0 (gi (D 0 )) ≥

i=1

n
X

di0 (gi (D)).

i=1

But, for E = D, D 0 , we have:
di0 (gi (E)) = di (gi (E)),

if i 6= j and d 0j (g j (E)) = t(g j (E)).

Therefore,
n
X

t(g j (D 0 )) − p j (D 0 ) +

di (gi (Z )) ≥ t(g j (D)) − p j (D) +

i=1,i6= j

n
X

di (gi (Z ))

i=1,i6= j

and t(g j (D 0 )) − p j (D 0 ) ≥ t(g j (D)) − p j (D).
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : Fintype Bidder] [inst_1 : DecidableEq Bidder]
  (alloc : EconCSLib.Auction.CombinatorialReport Bidder Item → EconCSLib.Auction.BundleAllocation Bidder Item),
  LOS02CombinatorialAuctions.ProofInterface.gvaWelfareMaximizingAllocationRule alloc →
    (LOS02CombinatorialAuctions.ProofInterface.generalizedVickreyAuction alloc).TruthfulDominantStrategy
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem4_1_generalized_vickrey_truthful
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 15

Source locator: cited publication, lines 385-423

Verbatim source input

```
and t(g j (D 0 )) − p j (D 0 ) ≥ t(g j (D)) − p j (D).
Notice that the second term in the payment of j does not depend on j’s declaration
and is therefore irrelevant to his decision on what to declare. A feature of the GVA
is that no truthful bidder’s utility can be negative.
PROPOSITION 4.2. If j is truthful, his utility u j in the GVA is nonnegative.
PROOF. By Eq. (3), since j is truthful, by Eq. (6), and finally by Eq. (4):
u j = d j (g j (D)) +
=

n
X
i=1

n
X

di (gi (D)) −

i=1,i6= j
n
X

di (gi (D)) −

n
X

di (gi (Z ))

i=1

di (gi (Z )) ≥ 0.

i=1

Since bidders truthfully declare their type and the allocation maximizes the sum
of the declared utilities, in a GVA, the allocation maximizes the sum of the true
valuations of the bidders, that is, the social welfare. In a quasilinear setting, this is
equivalent to Pareto optimality. Therefore, a GVA is Pareto optimal. The mechanism
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : Fintype Bidder] [inst_1 : DecidableEq Bidder]
  (alloc : EconCSLib.Auction.CombinatorialReport Bidder Item → EconCSLib.Auction.BundleAllocation Bidder Item),
  LOS02CombinatorialAuctions.ProofInterface.gvaWelfareMaximizingAllocationRule alloc →
    ∀ (values : EconCSLib.Auction.CombinatorialReport Bidder Item),
      LOS02CombinatorialAuctions.ProofInterface.nonnegativeValues values →
        ∀ (i : Bidder),
          0 ≤ (LOS02CombinatorialAuctions.ProofInterface.generalizedVickreyAuction alloc).utility values values i
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.proposition4_2_generalized_vickrey_truthful_utility_nonneg
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 16

Source locator: cited publication, lines 501-523

Verbatim source input

```
6. Infeasibility of the GVA
Let us now assume that all bidders are single-minded, that is, the set of all possible
types is now 6. It follows easily from Proposition 4.2 that, in a GVA, a singleminded bidder of type hs, vi never pays more than
↪  v and pays nothing if he is not
allocated the whole set s.
In a GVA, the allocation
Pn is the one defined in Eq. (4). Computing this allocation
di (a) over all a's in the set O that is of exponential size.
requires optimizing i=1
One may suspect that this an infeasible task. Indeed, the problem of finding the
allocation of Eq. (4) has been shown to be NP-hard in Rothkopf et al. [1998]. We
remark that the restriction to single-minded bidders does nothing to alleviate the
problem. Not only is it infeasible to solve exactly the optimization problem, but
it turns out to be also infeasible to guarantee any nontrivial approximation to the
optimal allocation. This follows easily from a celebrated result of Håstad [1999]
and was, independently, noticed by Sandholm to whom priority is due: final version
in Sandholm [2002].
THEOREM 6.1. Let a single-minded type di = hsi , v i i, si ⊆ G, v i ∈ R+ be given
for each bidder i ∈ P. Let
Pn|G| = k and |P| = n. The problem of finding an allodi (a) is NP-hard in k + n. Moreover, the existence
cation a that maximizes i=1
of a polynomial time algorithm guaranteed to find an allocation whose value is at
least k −1/2+² times the value of the optimal solution would imply that NP = ZPP.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (sets : Bidder → Finset Item) (weights : Bidder → ℝ) (selected : Finset Bidder),
  EconCSLib.Auction.PairwiseDisjointDesired
      (LOS02CombinatorialAuctions.ProofInterface.setPackingSingleMindedBids sets weights) selected ↔
    LOS02CombinatorialAuctions.ProofInterface.setPackingFeasible sets selected
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem6_1_set_packing_feasibility_encoding_correct
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 17

Source locator: cited publication, lines 501-523

Verbatim source input

```
6. Infeasibility of the GVA
Let us now assume that all bidders are single-minded, that is, the set of all possible
types is now 6. It follows easily from Proposition 4.2 that, in a GVA, a singleminded bidder of type hs, vi never pays more than
↪  v and pays nothing if he is not
allocated the whole set s.
In a GVA, the allocation
Pn is the one defined in Eq. (4). Computing this allocation
di (a) over all a's in the set O that is of exponential size.
requires optimizing i=1
One may suspect that this an infeasible task. Indeed, the problem of finding the
allocation of Eq. (4) has been shown to be NP-hard in Rothkopf et al. [1998]. We
remark that the restriction to single-minded bidders does nothing to alleviate the
problem. Not only is it infeasible to solve exactly the optimization problem, but
it turns out to be also infeasible to guarantee any nontrivial approximation to the
optimal allocation. This follows easily from a celebrated result of Håstad [1999]
and was, independently, noticed by Sandholm to whom priority is due: final version
in Sandholm [2002].
THEOREM 6.1. Let a single-minded type di = hsi , v i i, si ⊆ G, v i ∈ R+ be given
for each bidder i ∈ P. Let
Pn|G| = k and |P| = n. The problem of finding an allodi (a) is NP-hard in k + n. Moreover, the existence
cation a that maximizes i=1
of a polynomial time algorithm guaranteed to find an allocation whose value is at
least k −1/2+² times the value of the optimal solution would imply that NP = ZPP.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] (sets : Bidder → Finset Item) (weights : Bidder → ℝ)
  (selected : Finset Bidder),
  LOS02CombinatorialAuctions.paper_single_minded_total_value
      (LOS02CombinatorialAuctions.ProofInterface.setPackingSingleMindedBids sets weights) selected =
    LOS02CombinatorialAuctions.ProofInterface.weightedSetPackingValue weights selected
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem6_1_set_packing_value_encoding_correct
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 18

Source locator: cited publication, lines 540-548

Verbatim source input

```
PROOF. The problem at hand may be described as the weighted version of the
Set Packing problem of Karp [1972]. Karp shows that Set Packing is NP-hard by
reducing the Clique problem to it. The k used in this reduction is of the order of n 2 .
A direct reduction of Clique to our allocation problem is obtained in the following
way. Given a graph G, let the goods be the edges and the bids be the vertices. Each
vertex requests the edges it is adjacent to for a price of 1. An optimal allocation
is a maximal independent vertex set. Håstad [1999] has shown that Clique cannot
be approximated within |V |1−² unless NP = ZPP. The reduction mentioned above
shows our claim.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (sets : Bidder → Finset Item) (weights : Bidder → ℝ) (selected : Finset Bidder),
  LOS02CombinatorialAuctions.ProofInterface.weightedSetPackingOptimal sets weights selected ↔
    LOS02CombinatorialAuctions.ProofInterface.singleMindedOptimalAcceptedSet
      (LOS02CombinatorialAuctions.ProofInterface.setPackingSingleMindedBids sets weights) selected
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem6_1_weighted_set_packing_reduction
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 19

Source locator: cited publication, lines 540-548

Verbatim source input

```
PROOF. The problem at hand may be described as the weighted version of the
Set Packing problem of Karp [1972]. Karp shows that Set Packing is NP-hard by
reducing the Clique problem to it. The k used in this reduction is of the order of n 2 .
A direct reduction of Clique to our allocation problem is obtained in the following
way. Given a graph G, let the goods be the edges and the bids be the vertices. Each
vertex requests the edges it is adjacent to for a price of 1. An optimal allocation
is a maximal independent vertex set. Håstad [1999] has shown that Clique cannot
be approximated within |V |1−² unless NP = ZPP. The reduction mentioned above
shows our claim.
```

Expanded PaperInterface specification

```
∀ {Vertex : Type u_1} [inst : Fintype Vertex] [inst_1 : DecidableEq Vertex]
  (problem : LOS02CombinatorialAuctions.ProofInterface.graphCliqueDecisionInstance Vertex),
  LOS02CombinatorialAuctions.ProofInterface.graphCliqueDecisionProblem problem ↔
    LOS02CombinatorialAuctions.ProofInterface.singleMindedWelfareDecisionProblem
      (LOS02CombinatorialAuctions.ProofInterface.graphCliqueDecisionToSingleMindedWelfareDecision problem)
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem6_1_clique_decision_single_minded_welfare_reduction
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 20

Source locator: cited publication, lines 517-539

Verbatim source input

```
THEOREM 6.1. Let a single-minded type di = hsi , v i i, si ⊆ G, v i ∈ R+ be given
for each bidder i ∈ P. Let
Pn|G| = k and |P| = n. The problem of finding an allodi (a) is NP-hard in k + n. Moreover, the existence
cation a that maximizes i=1
of a polynomial time algorithm guaranteed to find an allocation whose value is at
least k −1/2+² times the value of the optimal solution would imply that NP = ZPP.


[form-feed in source extraction]
586

D. LEHMANN ET AL.

A short note on complexity classes: in the above, NP is the class of sets for which
membership can be decided nondeterministically in polynomial time and ZPP is
the subclass of NP consisting of those sets for which there is some constant c and a
probabilistic Turing machine M that on input x runs in expected time O(|x|c ) and
outputs 1 if and only if x ∈ L. The question of whether NP = ZPP is a deep open
question in theoretical computer science, related with the famous P = NP question.
NP = ZPP is not known to imply P = NP, but does imply NP = RP = co-RP =
co-NP. P = NP obviously implies NP = ZPP. RP is the class of sets for which
membership can be decided in polynomial time by a randomizing algorithm. The
class co-RP is the class of sets whose complements are in RP: nonmembership can
be decided polynomially by a randomizing algorithm. Similarly for co-NP. ZPP is
the intersection of RP and co-RP. (end of short note).
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} {Language : Type u_3} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (complexityModel : EconCSLib.Complexity.ComplexityClassModel Language)
  (FeasibleSM : ((Bidder → EconCSLib.Auction.SingleMindedBid Item) → Finset Bidder) → Prop)
  (FeasibleWSP : ((Bidder → Finset Item) → (Bidder → ℝ) → Finset Bidder) → Prop)
  (solver : (Bidder → EconCSLib.Auction.SingleMindedBid Item) → Finset Bidder),
  LOS02CombinatorialAuctions.ProofInterface.singleMindedOptimalSolver solver →
    FeasibleSM solver →
      (FeasibleSM solver →
          FeasibleWSP (LOS02CombinatorialAuctions.ProofInterface.setPackingSolverOfSingleMindedSolver solver)) →
        (∀ (wspSolver : (Bidder → Finset Item) → (Bidder → ℝ) → Finset Bidder),
            LOS02CombinatorialAuctions.ProofInterface.weightedSetPackingOptimalSolver wspSolver →
              FeasibleWSP wspSolver → complexityModel.npEqZPP) →
          complexityModel.npEqZPP
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem6_1_external_optimal_solver_np_eq_zpp
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 21

Source locator: cited publication, lines 517-539

Verbatim source input

```
THEOREM 6.1. Let a single-minded type di = hsi , v i i, si ⊆ G, v i ∈ R+ be given
for each bidder i ∈ P. Let
Pn|G| = k and |P| = n. The problem of finding an allodi (a) is NP-hard in k + n. Moreover, the existence
cation a that maximizes i=1
of a polynomial time algorithm guaranteed to find an allocation whose value is at
least k −1/2+² times the value of the optimal solution would imply that NP = ZPP.


[form-feed in source extraction]
586

D. LEHMANN ET AL.

A short note on complexity classes: in the above, NP is the class of sets for which
membership can be decided nondeterministically in polynomial time and ZPP is
the subclass of NP consisting of those sets for which there is some constant c and a
probabilistic Turing machine M that on input x runs in expected time O(|x|c ) and
outputs 1 if and only if x ∈ L. The question of whether NP = ZPP is a deep open
question in theoretical computer science, related with the famous P = NP question.
NP = ZPP is not known to imply P = NP, but does imply NP = RP = co-RP =
co-NP. P = NP obviously implies NP = ZPP. RP is the class of sets for which
membership can be decided in polynomial time by a randomizing algorithm. The
class co-RP is the class of sets whose complements are in RP: nonmembership can
be decided polynomially by a randomizing algorithm. Similarly for co-NP. ZPP is
the intersection of RP and co-RP. (end of short note).
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} {Language : Type u_3} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (factor : ℝ) (complexityModel : EconCSLib.Complexity.ComplexityClassModel Language)
  (FeasibleSM : ((Bidder → EconCSLib.Auction.SingleMindedBid Item) → Finset Bidder) → Prop)
  (FeasibleWSP : ((Bidder → Finset Item) → (Bidder → ℝ) → Finset Bidder) → Prop)
  (solver : (Bidder → EconCSLib.Auction.SingleMindedBid Item) → Finset Bidder),
  LOS02CombinatorialAuctions.ProofInterface.singleMindedApproximationSolverAtLeast factor solver →
    FeasibleSM solver →
      (FeasibleSM solver →
          FeasibleWSP (LOS02CombinatorialAuctions.ProofInterface.setPackingSolverOfSingleMindedSolver solver)) →
        (∀ (wspSolver : (Bidder → Finset Item) → (Bidder → ℝ) → Finset Bidder),
            LOS02CombinatorialAuctions.ProofInterface.weightedSetPackingApproximationSolverAtLeast factor wspSolver →
              FeasibleWSP wspSolver → complexityModel.npEqZPP) →
          complexityModel.npEqZPP
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem6_1_external_approximation_solver_np_eq_zpp
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 22

Source locator: cited publication, lines 528-539

Verbatim source input

```
A short note on complexity classes: in the above, NP is the class of sets for which
membership can be decided nondeterministically in polynomial time and ZPP is
the subclass of NP consisting of those sets for which there is some constant c and a
probabilistic Turing machine M that on input x runs in expected time O(|x|c ) and
outputs 1 if and only if x ∈ L. The question of whether NP = ZPP is a deep open
question in theoretical computer science, related with the famous P = NP question.
NP = ZPP is not known to imply P = NP, but does imply NP = RP = co-RP =
co-NP. P = NP obviously implies NP = ZPP. RP is the class of sets for which
membership can be decided in polynomial time by a randomizing algorithm. The
class co-RP is the class of sets whose complements are in RP: nonmembership can
be decided polynomially by a randomizing algorithm. Similarly for co-NP. ZPP is
the intersection of RP and co-RP. (end of short note).
```

Expanded PaperInterface specification

```
∀ {Language : Type u_1}
  (complexityModel : LOS02CombinatorialAuctions.ProofInterface.randomizedComplexityClassModel Language),
  complexityModel.npEqZPP →
    complexityModel.NP = complexityModel.RP ∧
      complexityModel.NP = complexityModel.coRP ∧ complexityModel.NP = complexityModel.coNP
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.complexity_note_np_eq_zpp_implies_randomized_collapse
```

Recorded source-to-Spec screening **Verdict:** not recorded
Reason: not recorded
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 23

Source locator: cited publication, lines 653-740

Verbatim source input

```
THEOREM 7.2. The greedy allocation√scheme with norm a/|s|1/2 approximates
the optimal allocation within a factor of k.
PROOF. Assume the bids (i.e.,
√ bidders) are hsi , ai i for i = 1, . . . , n. Let
w i = |si |. Our norm is: ri = ai / w i . Let OP be the optimal solution, that is, the
set ofPbids contained in the optimal solution. The value of the optimal solution is
ai . Let GR be the solution obtained by the greedy allocation and β its
α = i∈OP P
value: β = i∈GR ai . We want to show that:
√
α ≤ β k.
(7)
Notice, first, that we may, without loss of generality, assume that the sets OP and
GR have no bid in common. Indeed, if they have, one considers the problem in
which the common bids and all the units they request have been removed. The
greedy and optimal solutions of the new problem are similar to the old ones and the
inequality for the new smaller problem implies the same for the original problem.
Let us consider β. By elementary algebraic considerations:
rX
rX
X
ai ≥
ai 2 =
ri 2 w i .
β=
i∈GR

i∈GR

i∈GR

Consider α. By the Cauchy–Schwarz inequality:
rX
rX
X √
ri w i ≤
ri 2
wi .
α=
P

i∈OP

i∈OP

i∈OP

The expression i∈OP w i represents the total number of goods allocated in the
optimal allocation OP and is therefore bounded from above by k, the number of


[form-feed in source extraction]
Approximately Efficient Combinatorial Auctions
goods available. We conclude that:
α≤

rX

ri 2

589

√
k.

i∈OP

To prove (7), it will be enough, then, to prove that:
X
X
ri 2 ≤
ri 2 w i .
i∈OP

i∈GR

Consider the optimal solution OP. By assumption, the bids of OP did not enter
the greedy solution GR. This means that, at the time any such bid i is considered
during the execution of the greedy algorithm, it cannot be entered in the partial
allocation already built. This implies that there is a good l ∈ si that has already
been allocated in the partial greedy solution, that is, there is a bid j in GR, with
r j ≥ ri and l ∈ s j .
A number of different bids from OP may, in this way, be associated with the
same bid j of GR, but at most w j different bids of OP may be associated with bid j
of GR, since the sets of goods requested by two different bids of OP have an empty
intersection. If OP j is the set of bids of OP that are associated with bid j:
X
ri 2 ≤ r j 2 w j .
i∈OP j
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item] [Inhabited Bidder]
  [Inhabited Item] (bids : Bidder → EconCSLib.Auction.SingleMindedBid Item) (optimal : Finset Bidder)
  (goods : Finset Item) (order : List Bidder),
  (∀ i ∈ optimal, (bids i).desired ⊆ goods) →
    EconCSLib.Auction.PairwiseDisjointDesired bids optimal →
      (∀ i ∈ optimal, Finset.Nonempty (bids i).desired) →
        (∀ i ∈ LOS02CombinatorialAuctions.ProofInterface.greedyAcceptedFromOrder bids order,
            Finset.Nonempty (bids i).desired) →
          (∀ i ∈ optimal, 0 ≤ (bids i).value) →
            (∀ i ∈ LOS02CombinatorialAuctions.ProofInterface.greedyAcceptedFromOrder bids order, 0 ≤ (bids i).value) →
              EconCSLib.Auction.SingleMindedSqrtNormDescending bids order →
                (∀ i ∈ optimal, i ∈ order) →
                  LOS02CombinatorialAuctions.ProofInterface.singleMindedTotalValue bids optimal ≤
                    √↑goods.card *
                      LOS02CombinatorialAuctions.ProofInterface.singleMindedTotalValue bids
                        (LOS02CombinatorialAuctions.ProofInterface.greedyAcceptedFromOrder bids order)
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem7_2_sqrt_norm_approx_of_sorted_order
```

Recorded source-to-Spec screening **Verdict:** `not recorded`

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 24

Source locator: cited publication, lines 878-887

Verbatim source input

```
LEMMA 9.1. In a mechanism that satisfies Exactness and Monotonicity, given
a bidder j, a set s of goods and declarations for all other bidders, there exists a
critical value v c such that
∀v, v < v c ⇒ g j = ∅,
∀v, v > v c ⇒ g j = s,
We allow v c to be infinite if f (As,v )−1 ( j) = ∅ for every v. Note that we do not
know whether j's bid is granted or not in case v = v c .
PROOF. By Monotonicity, the set of v's for which g j = ∅ is empty (in which
case take v c = 0), a semi-open set of the form [0, v c [ or a closed set of the form
[0, v c ] or equal to R+ .
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item),
  M.MonotonicityOn LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile →
    ∀ (reports : Bidder → EconCSLib.Auction.SingleMindedBid Item),
      LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile reports →
        ∀ (i : Bidder) (s : Finset Item),
          s.Nonempty →
            (∃ c,
                0 ≤ c ∧
                  (∀ (v : ℝ), 0 ≤ v → v < c → i ∉ M.accepted (Function.update reports i { desired := s, value := v })) ∧
                    ∀ (v : ℝ),
                      0 ≤ v → c < v → i ∈ M.accepted (Function.update reports i { desired := s, value := v })) v
              ∀ (v : ℝ), 0 ≤ v → i ∉ M.accepted (Function.update reports i { desired := s, value := v })
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.lemma9_1_exists_nonnegative_critical_value_of_monotonicity
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 25

Source locator: cited publication, lines 922-927

Verbatim source input

```
Any mechanism that satisfies the conditions above is truthful. A number of
preliminary lemmas are needed.
LEMMA 9.2. In a mechanism that satisfies Exactness and Participation, a
bidder whose bid is denied has utility zero.
PROOF. By Exactness, the bidder gets nothing and his valuation is zero. By
Participation, his payment is zero.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item),
  M.Participation →
    ∀ (values reports : Bidder → EconCSLib.Auction.SingleMindedBid Item),
      LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile values →
        ∀ {i : Bidder}, i ∉ M.accepted reports → M.utility values reports i = 0
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.lemma9_2_denied_bidder_utility_eq_zero
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 26

Source locator: cited publication, lines 928-932

Verbatim source input

```
LEMMA 9.3. In a mechanism that satisfies Exactness, Monotonicity, Participation and Critical a truthful bidder’s utility is
↪  nonnegative.
PROOF. If j’s bid is denied, we conclude by Lemma 9.2. Assume j’s bid is
granted and his type is hs, vi. Since he is truthful, his declaration is d j = hs, vi. We
conclude that j is allocated s and his valuation is v. By Lemma 9.1, since j’s bid
is granted, v ≥ v c . By Critical, j’s payment is v c , and his utility is v − v c ≥ 0.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item),
  M.Participation →
    ∀ (C : M.NonnegativeCriticalValueWithInfinityCertificate)
      (values : Bidder → EconCSLib.Auction.SingleMindedBid Item),
      LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile values →
        ∀ (i : Bidder), 0 ≤ M.utility values values i
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.lemma9_3_truthful_utility_nonnegative_condition
```

Recorded source-to-Spec screening **Verdict:** `not recorded`

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 27

Source locator: cited publication, lines 933-945

Verbatim source input

```
The next lemma shows that a bidder cannot benefit from lying just about his
value (he truthfully declares the set of goods he is interested in).
LEMMA 9.4. In a mechanism that satisfies Exactness, Monotonicity, Participation and Critical, a bidder j of type hs, vi is never
↪  better off declaring hs, v 0 i for
some v 0 6= v than by being truthful.
PROOF. Compare the case j bids, truthfully, hs, vi and the case he bids hs, v 0 i.
Let g j be the bundle he gets in the first case and g 0j the bundle he gets in the second
case. If j's bid is denied in the second case (i.e., if g 0j 6= s), then, by Lemma 9.2,
his utility is zero in the second case and, by Lemma 9.3, his utility in the first case
is nonnegative. The claim holds.
Assume therefore that g 0j = s. If both bids are granted, j has the same valuation
(v) and pays the same payment, v c (by Critical). If g 0j = s but g j = ∅, it must be
the case that v ≤ v c ≤ v 0 . Being truthful gives j, by Lemma 9.2, zero utility. Lying
gives him utility v − v c ≤ 0.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item),
  M.MonotonicityOn LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile →
    M.Participation →
      ∀ (C : M.NonnegativeCriticalValueWithInfinityCertificate)
        (values : Bidder → EconCSLib.Auction.SingleMindedBid Item),
        LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile values →
          ∀ (i : Bidder) {v' : ℝ},
            0 ≤ v' →
              M.utility values (Function.update values i { desired := (values i).desired, value := v' }) i ≤
                M.utility values values i
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.lemma9_4_no_profitable_value_only_lie_of_nonnegative_infinity_axioms
```

Recorded source-to-Spec screening **Verdict:** `not recorded`

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 28

Source locator: cited publication, lines 946-952

Verbatim source input

```
LEMMA 9.5. In a mechanism that satisfies Exactness, Monotonicity and Critical, a bidder j declaring type hs, vi whose bid is
↪  granted, that is, g j = s, pays a
price p j that is at least the price p 0j that he would have paid had he declared his
type as hs 0 , vi for any s 0 ⊆ s.
PROOF. By Monotonicity, the bid hs 0 , vi would have been granted and by Critical, the price p 0j paid for such a bid satisfies:
↪  for any x < p 0j the bid hs 0 , xi would
not have been granted. By Monotonicity, for any such x the bid hs, xi would not
have been granted. By Critical, for any x such that x > p j , the bid hs, xi would
have been granted. We conclude that p 0j ≤ p j .
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item),
  M.MonotonicityOn LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile →
    ∀ (C : M.NonnegativeCriticalValueWithInfinityCertificate)
      (reports : Bidder → EconCSLib.Auction.SingleMindedBid Item),
      LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile reports →
        ∀ (i : Bidder) {sSmall sLarge : Finset Item} {pLarge : ℝ},
          sSmall.Nonempty →
            sLarge.Nonempty →
              sSmall ⊆ sLarge →
                C.threshold reports i sLarge = some pLarge →
                  ∃ pSmall, C.threshold reports i sSmall = some pSmall ∧ pSmall ≤ pLarge
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.lemma9_5_finite_threshold_mono_of_nonnegative_infinity_certificate
```

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 29

Source locator: cited publication, lines 954-968

Verbatim source input

```
THEOREM 9.6. If a mechanism satisfies Exactness, Monotonicity, Participation
and Critical, then it is a truthful mechanism.
PROOF. Suppose j's type is hs, vi. Could j have any interest in declaring his
type as hs 0 , v 0 i? By Lemma 9.3, the only case we have to consider is when declaring
hs 0 , v 0 i j gets a positive utility, and by Lemma 9.2 this means that j's bid is granted.
Assume, therefore that g 0j = s 0 . If s 6⊆ s 0 , the valuation of j is zero. Since, by


[form-feed in source extraction]
594

D. LEHMANN ET AL.

Critical, his payment is non-negative, his utility cannot be positive. Assume then
s ⊆ s 0 . Since j's valuation for s 0 is the same as for s, Lemma 9.5 implies that,
instead of declaring hs 0 , v 0 i, j would not have been worse off by declaring hs, v 0 i.
Lemma 9.4 implies that declaring hs, v 0 i cannot be better than being truthful.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : DecidableEq Bidder] [inst_1 : DecidableEq Item]
  (M : EconCSLib.Auction.SingleMindedAcceptedMechanism Bidder Item),
  M.MonotonicityOn LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile →
    M.Participation →
      ∀ (C : M.NonnegativeCriticalValueWithInfinityCertificate),
        LOS02CombinatorialAuctions.ProofInterface.singleMindedTruthfulOn M
          LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem9_6_single_minded_truthful_of_nonnegative_infinity_axioms
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 30

Source locator: cited publication, lines 969-1020

Verbatim source input

```
10. A Truthful Mechanism with Greedy Allocation
We shall now describe the payment mechanism that we propose to be used in
conjunction with the greedy allocation of Section 7. The description of the payments
is tightly linked with that of the greedy algorithm. The computation of the payment
is performed in parallel with the execution of the greedy algorithm and takes time
linear in the number of bidders for each payment. On the whole, computing the
allocation and the payments takes time at most quadratic in the number of bids.
We assume that the criterion used is average amount per good, the adaptation
to most other suitable greedy allocations is obvious. Informally, a bidder pays, per
good, the average price proposed by the first bid in the list L that is denied because
of this bid. Consider a bid j in L. Let c( j) be the average amount per good of j. We
shall denote by n( j) the first bid following j (bids are sorted in decreasing order,
that is, c( j) ≥ c(n( j))) that has been denied but would have been granted were it
not for the presence of j. Assume that such a bid exists. Notice that such a bid
necessarily conflicts with j, and therefore:
n( j) = min{i | j < i, s( j) ∩ s(i) 6= ∅, ∀l, l < i, l 6= j, l granted ⇒ s(l) ∩ s(i) = ∅}.
Definition 10.1 Greedy Payment Scheme.
the first phase.

Let L be the sorted list obtained in

– j pays zero if his bid is denied or if there is no bid n( j),
–if there is an n( j) and j's bid hs, vi is granted, he pays |s| × c(n( j)).
We may now state the main result of this article.
THEOREM 10.2. The mechanism composed of the greedy allocation and payment schemes is truthful for single-minded bidders.
PROOF. We shall prove that greedy mechanism satisfies Exactness, Monotonicity, Participation and Critical and use Theorem 9.6.
↪  The description of the greedy
allocation scheme makes it clear that every bid is either granted or denied. The
greedy allocation satisfies Exactness. For Monotonicity, assume that s ⊆ s 0 and
that v ≥ v 0 and let c be the norm of hs, vi and c0 the norm of hs 0 , v 0 i. By our
assumption concerning norms we have c ≥ c0 . If we compare the list L and L 0 obtained, respectively, we see that, since there
↪  are no ties by assumption, they differ
only in that j's bid may have been moved backwards by the change from hs, vi to
hs 0 , v 0 i. The greedy allocation algorithm performs, that is, grants or denies bids, in
exactly the same way on L and L 0 until it gets to j's bid in L. Assume j's bid is
denied in L: there is some bid that conflict with it that has been granted already.
The same bid also conflicts with j's bid in L 0 since s ⊆ s 0 and this bid will also be
denied. Similarly, if j's bid in L 0 is granted, no bid granted before conflicts with it
and therefore no bid granted before j's in L conflicts with it either and j's bid is also
granted in L. We have shown that the greedy allocation satisfies Monotonicity. It is


[form-feed in source extraction]
Approximately Efficient Combinatorial Auctions

595

clear from the first part of Definition 10.1 that it satisfies Participation. For Critical,
notice that the second part of Definition 10.1 defines the payment for a bid granted
at exactly the minimal declared value that would have allowed it to be granted, v c .
Any declared value above |s| × c(n( j)) leaves j before n( j). If there was a bid i,
j < i < n( j) that would prevent the granting of j displaced in such a way, i would
have to be granted and conflict with j. It is therefore a bid denied in the original
allocation, that would have been granted were it not for j, contradicting the fact
that n( j) is the first such bid. Any declared value below |s| × c(n( j)) guarantees
the denial of j because n( j) is granted.
```

Expanded PaperInterface specification

```
∀ {Bidder : Type u_1} {Item : Type u_2} [inst : Fintype Bidder] [inst_1 : DecidableEq Item]
  [inst_2 : LinearOrder Bidder],
  LOS02CombinatorialAuctions.ProofInterface.singleMindedTruthfulOn
    LOS02CombinatorialAuctions.ProofInterface.averageGreedyAcceptedMechanism
    LOS02CombinatorialAuctions.ProofInterface.nonnegativeNonemptySingleMindedProfile
```

Lean proof endpoint (built separately)

```
LOS02CombinatorialAuctions.PaperInterface.theorem10_2_averageGreedy_truthful
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation